סוכני בינה מלאכותית
עם פרוטוקול הקשר המודל

AI Agents with Model Context Protocol

מאת

ד"ר יורם סגל

December 9, 2025

תוכן העניינים

1 הקדמה **3**

1.1 בשער המהפכה הטכנולוגית 3

1.2 המעבר מהבנה לפעולה 3

1.3 פרוטוקול הקשר המודל 3

1.4 למי מיועד הספר 3

1.5 מבנה הספר 3

1.6 תודות 4

I יסודות **4**

2 מבוא לבינה מלאכותית אגנטית **5**

2.1 פתיחה: מחשבה על המחשבה הפועלת 5

2.2 הגדרה פורמלית של סוכן 5

2.2.1 מודל מתמטי של סוכן 5

2.2.2 מחזור הפעולה הסוכני 6

2.2.3 סוכנים מבוססי LLM 6

2.3 זרימות עבודה אגנטיות לעומת סוכנים אוטונומיים 6

2.3.1 זרימת עבודה אגנטית 6

2.3.2 סוכן אוטונומי מלא 7

2.4 ארכיטקטורת הליבה של סוכן LLM 7

2.4.1 רכיב 1: מודל השפה — המוח החושב 7

2.4.2 רכיב 2: כלים — הידיים הפועלות 7

2.4.3 רכיב 3: לולאת הסוכן — הלב הפועם 8

2.5 האתגר המרכזי: חיבור סוכנים לעולם 8

2.5.1 הבעיה: פרגמנטציה ואי-תאימות 8

2.5.2 הפתרון: פרוטוקול סטנדרטי 8

2.6 סיכום הפרק והצעד הבא 8

3 פרוטוקול הקשר המודל — סקירה מקיפה **9**

3.1 על פרוטוקולים וסטנדרטים בהיסטוריה 9

3.2 מהי הבעיה ש-MCP פותר 9

3.3 ארכיטקטורת שלוש השכבות 10

3.3.1 יישום מארח — Host Application 10

3.3.2 לקוח MCP — MCP Client 11

3.3.3 שרת MCP — MCP Server 12

3.4 שכבות התעבורה — Transport Layers 12

3.4.1 תעבורת קלט/פלט סטנדרטי — stdio Transport 12
3.4.2 תעבורת HTTP זורמת — Streamable HTTP Transport 13
3.4.3 השוואת שכבות התעבורה 14
3.5 פרוטוקול התקשורת — JSON-RPC 14
3.5.1 מדוע JSON-RPC? 14
3.5.2 מבנה ההודעות 15
3.5.3 זרימת הודעות דו-כיוונית 17
3.6 מחזור חיי החיבור — Connection Lifecycle 17
3.6.1 מכונת המצבים של החיבור 17
3.6.2 שלב האתחול — Initialization Phase 18
3.6.3 שלב הפעולה — Active Phase 19
3.6.4 שלב הסיום — Termination Phase 20
3.7 הפרימיטיבים של MCP 20
3.7.1 כלים — פונקציות לביצוע 21
3.7.2 משאבים — מידע לקריאה 21
3.7.3 הנחיות — תבניות מוכנות 22
3.8 סיכום הפרק 23

II לקוחות MCP 23

4 ארכיטקטורת לקוח MCP 25
4.1 מודל לקוח-שרת בהיסטוריה של המחשוב 25
4.2 מודל פורמלי של לקוח MCP 25
4.2.1 הגדרת מצב הלקוח 25
4.2.2 פונקציות המעבר 26
4.3 רכיבי הליבה של לקוח MCP 26
4.3.1 מנהל הסשנים 26
4.3.2 רישום הכלים 26
4.3.3 מטפל התעבורה 27
4.4 ארכיטקטורה פנימית של הלקוח 27
4.5 טופולוגיית חיבור רב-שרתית 27
4.5.1 מודל חיבור מרובה 28
4.5.2 ניהול מרחב שמות 28
4.6 זרימת הודעות בין לקוח לשרתים 28
4.6.1 מחזור חיי בקשה-תשובה 29
4.7 יישום לקוח בסיסי ב-Python 29
4.7.1 יצירת סשן יחיד 30
4.7.2 מחלקת לקוח רב-שרתי 32

© Dr. Segal Yoram - כל הזכויות שמורות

4.8 טיפול בשגיאות ועמידות 35
4.8.1 סוגי שגיאות נפוצות 35
4.8.2 מנגנון ניסיונות חוזרים 35
4.9 חלוקת אחריות: לקוח מול שרת 37
4.10 אופטימיזציה וביצועים 37
4.10.1 מטמון תשובות 37
4.10.2 ביצוע מקבילי 39
4.11 סיכום הפרק 39

5 בניית לקוחות MCP — מדריך מעשי 41
5.1 על בונים ומשתמשים 41
5.2 הקמת סביבת הפיתוח 41
5.2.1 התקנת Python SDK 41
5.2.2 מבנה הפרויקט המומלץ 42
5.3 לקוח מינימלי — הצעד הראשון 42
5.4 הפעלת כלים — הליבה האמיתית 44
5.5 תרשים רצף של אתחול לקוח 46
5.6 לולאת הסוכן המלאה 46
5.7 טיפול בתעבורת HTTP 48
5.8 ניהול קונפיגורציה 50
5.9 דוגמה מלאה: לקוח CLI מתוחכם 52
5.10 טיפול בשגיאות ונתיבי התאוששות 54
5.11 סיכום הפרק 56

6 יכולות מתקדמות בלקוח MCP 57
6.1 על המורכבות והתוחכה 57
6.2 דגימה — כאשר השרת מבקש ממך לחשוב 57
6.2.1 מתי משתמשים בדגימה? 57
6.2.2 הזרימה הדו-כיוונית 58
6.2.3 מימוש דגימה בלקוח 59
6.3 ניהול מרובה שרתים 60
6.3.1 טופולוגיית רב-שרתים 60
6.3.2 מחלקת ניהול חיבורים 61
6.4 מנויים למשאבים 62
6.4.1 מימוש מערכת מנויים 63
6.5 התראות התקדמות 64
6.6 אסטרטגיות התאוששות משגיאות 66
6.6.1 טקסונומיה של שגיאות 66
6.6.2 מודל מתמטי לניסיונות חוזרים 66

6.7 איזון עומסים בין שרתים 68
6.7.1 מודל מתמטי לאיזון עומסים 68
6.8 דוגמה מלאה: לקוח מתקדם 70
6.9 סיכום הפרק 72

III שרתי MCP 72

7 ארכיטקטורת שרת MCP 73
7.1 שרתים לאורך ההיסטוריה: מארכיונים עתיקים ועד שרתי MCP 73
7.2 עקרונות התכנון של שרת MCP 73
7.2.1 עקרון האחריות הבודדת 73
7.2.2 עקרון הממשק הברור 74
7.2.3 עקרון האבטחה 74
7.3 ארכיטקטורה פנימית של שרת MCP 74
7.3.1 שכבת התעבורה 74
7.3.2 שכבת הפרוטוקול 75
7.3.3 מטפלי פרימיטיבים 75
7.3.4 שכבת הלוגיקה 75
7.4 צינור עיבוד בקשות 75
7.5 רישום הפרימיטיבים 76
7.6 מודל מתמטי לניהול קיבולת שרת 77
7.7 שרתים מקומיים מול שרתים מרוחקים 78
7.8 מחזור החיים של שרת MCP 78
7.8.1 שלב 1: אתחול 78
7.8.2 שלב 2: פעולה 79
7.8.3 שלב 3: סיום 79
7.9 שיקולי אבטחה 80
7.9.1 אימות קלט 80
7.9.2 הגבלת קצב 81
7.10 סיכום הפרק 83

8 בניית שרתי MCP — מבראשית ועד השלמה 84
8.1 על מעשה הבריאה ואומנות הבנייה 84
8.2 היסוד הראשון — שרת מינימלי 84
8.2.1 קוד היסוד 84
8.2.2 ניתוח רכיבי היסוד 85
8.2.3 תהליך האתחול 86
8.3 שכבת הכלים — מתן יכולת לפעול 86
8.3.1 שרת עם כלי בסיסי 87

כל הזכויות שמורות - © Dr. Segal Yoram

8.3.2 אנטומיה של כלי 88
8.4 בניית שרת מערכת קבצים 88
8.4.1 עקרונות אבטחה בשרת קבצים 90
8.5 הוספת משאבים — חשיפת מידע 90
8.6 הוספת הנחיות — תבניות מוכנות 92
8.7 דיאגרמת ארכיטקטורה מלאה 94
8.8 מדריך צעד אחר צעד 94
8.8.1 שלב 1: הקמת הפרויקט 94
8.8.2 שלב 2: הוספת כלי ראשון 95
8.8.3 שלב 3: בדיקת השרת 95
8.8.4 שלב 4: הפעלה ואינטגרציה 97
8.9 שיטות עבודה מומלצות 97
8.9.1 תיעוד מקיף 97
8.9.2 טיפול חכם בשגיאות 97
8.9.3 רישום ומעקב 97
8.9.4 בדיקות אוטומטיות 97
8.10 סיכום — מבראשית ועד השלמה 97

9 פרימיטיבים של שרת — אבני הבניין של האינטליגנציה 99
9.1 מאטומים למולקולות: פילוסופיה של פרימיטיבים 99
9.2 כלים — הידיים שמשנות את העולם 99
9.2.1 טבע הכלי 99
9.2.2 אנטומיה של כלי 100
9.2.3 מתמטיקה של ולידציית סכמה 101
9.2.4 מחזור החיים של כלי 101
9.2.5 מימוש כלי בפועל 101
9.3 משאבים — העיניים שרואות את המידע 103
9.3.1 מהות המשאב 103
9.3.2 אנטומיה של משאב 103
9.3.3 תבניות משאבים 104
9.3.4 מחזור החיים של משאב 104
9.3.5 מנויים למשאבים דינמיים 104
9.4 הנחיות — המורים שמעבירים ידע 107
9.4.1 טבע ההנחיה 107
9.4.2 אנטומיה של הנחיה 107
9.4.3 יצירת הנחיות דינמיות 108
9.4.4 מחזור החיים של הנחיה 110
9.5 השוואה בין הפרימיטיבים 110
9.6 שילוב הפרימיטיבים: מקרה מבחן 110

9.7 עקרונות תכנון פרימיטיבים 112
9.7.1 עקרון האחריות היחידה 112
9.7.2 עקרון השמות המתארים 112
9.7.3 עקרון התיאורים המפורטים 112
9.8 סיכום הפרק 113

VI תעבורה ואינטגרציה 113

10 שכבות התעבורה — stdio ו-HTTP 114
10.1 אבולוציה של תקשורת: מאותות עשן ועד זרמי נתונים 114
10.2 אנטומיה של שכבת תעבורה 114
10.2.1 שלוש השאלות היסודיות 114
10.2.2 מודל ההפשטה 115
10.3 תעבורת stdio — הצינורות המקומיים 115
10.3.1 צינורות בהיסטוריה של יוניקס 115
10.3.2 איך stdio עובד ב-MCP 116
10.3.3 יתרונות תעבורת stdio 116
10.3.4 חסרונות תעבורת stdio 117
10.3.5 מימוש לקוח stdio 117
10.3.6 מימוש שרת stdio 118
10.4 תעבורת HTTP זורמת — הרשת הגלובלית 120
10.4.1 האתגר של תקשורת דו-כיוונית ב-HTTP 120
10.4.2 ארכיטקטורת שני הערוצים 120
10.4.3 יתרונות תעבורת HTTP 121
10.4.4 חסרונות תעבורת HTTP 121
10.4.5 מימוש לקוח HTTP 122
10.4.6 מימוש שרת HTTP עם FastAPI 123
10.5 מודל מתמטי: סדר וערבות מסירה 124
10.5.1 הגדרות פורמליות 124
10.5.2 ניתוח התעבורות 124
10.6 טבלת השוואה מקיפה 125
10.7 שיקולי אבטחה 125
10.7.1 אבטחת stdio 125
10.7.2 אבטחת HTTP 126
10.8 קווים מנחים לבחירת תעבורה 127
10.8.1 מתי לבחור stdio 127
10.8.2 מתי לבחור HTTP 127
10.9 סיכום הפרק 127

כל הזכויות שמורות - © Dr. Segal Yoram

11 דפוסי אינטגרציה ומקרי שימוש 129
11.1 מבוא: מאבני בניין לאקוסיסטם 129
11.2 דפוס: סוכן עם כלים מקומיים 129
11.2.1 ארכיטקטורה 129
11.2.2 מימוש 130
11.2.3 מתי להשתמש 131
11.3 דפוס: שרת ארגוני מרכזי 131
11.3.1 ארכיטקטורה 131
11.3.2 מימוש 132
11.3.3 יתרונות וחסרונות 133
11.4 דפוס: שרשרת שרתים 133
11.4.1 ארכיטקטורה 133
11.4.2 מימוש 134
11.5 דפוס: סוכן רב-מודלי 135
11.5.1 ארכיטקטורה 135
11.5.2 מימוש 136
11.6 דפוס: ניטור וביקורת 137
11.6.1 מימוש 138
11.7 מקרי שימוש בעולם האמיתי 139
11.7.1 עוזר פיתוח (Dev Assistant) 139
11.7.2 עוזר מחקר (Research Assistant) 139
11.7.3 אוטומציה עסקית (Business Automation) 140
11.8 שיטות עבודה מומלצות 141
11.8.1 עקרונות ארכיטקטוניים 141
11.8.2 אבטחה והרשאות 141
11.8.3 ביצועים וסקלביליות 141
11.9 סיכום הפרק והספר 141
11.9.1 סיכום הפרק 141
11.9.2 מסע בעולם MCP — סיכום הספר 142

12 דוגמאות קוד מלאות 146
12.1 לקוח MCP מלא 146
12.2 שרת MCP מלא 147
12.3 קובץ קונפיגורציה 148
12.4 שרת מסד נתונים מתקדם 148
12.5 לקוח עם ממשק שורת פקודה 150
12.6 בדיקות אוטומטיות 152
12.7 פריסה עם Docker 154
12.8 שרת HTTP עם SSE 156

12.9 לקוח HTTP 158
12.10 ניטור וניפוי שגיאות 160

13 מקורות ומשאבים 162
13.1 תיעוד רשמי 162
13.1.1 פרוטוקול MCP 162
13.1.2 SDKים רשמיים 162
13.1.3 תיעוד יישומים תומכים 162
13.2 שרתים מוכנים 163
13.2.1 שרתים רשמיים 163
13.2.2 שרתי קהילה 164
13.2.3 כתיבת שרת משלכם 164
13.3 יישומים תומכים 165
13.3.1 יישומי שולחן עבודה 165
13.3.2 כלי שורת פקודה 165
13.3.3 ספריות וכלי פיתוח 165
13.4 ספרות נוספת 166
13.4.1 ספר המקור 166
13.4.2 מאמרים אקדמיים 166
13.4.3 בלוגים ופוסטים 166
13.4.4 קורסים מקוונים 167
13.5 קהילה ותמיכה 167
13.5.1 פורומים ודיונים 167
13.5.2 דיווח על באגים ובקשות לתכונות 167
13.5.3 תרומה לפרויקט 167
13.6 כלי פיתוח 168
13.6.1 בדיקות וניפוי באגים 168
13.6.2 תבניות פרויקט 168
13.6.3 כלי אבטחה 168
13.6.4 כלי ניטור וביצועים 168
13.7 מילון מונחים מקיף 169
13.7.1 מונחי יסוד 169
13.7.2 ארכיטקטורה 169
13.7.3 פרימיטיבים 169
13.7.4 תקשורת ופרוטוקול 170
13.7.5 יכולות מתקדמות 170
13.7.6 אבטחה והרשאות 170
13.7.7 פיתוח ובדיקות 171
13.7.8 מונחים טכניים נוספים 171

כל הזכויות שמורות - © Dr. Segal Yoram

13.8 סיכום 172

ביבליוגרפיה **172**

1 הקדמה

1.1 בשער המהפכה הטכנולוגית

בשער העידן החדש, עומדים אנו כנוכח תמורה שאינה פחותה במשמעותה ממעבר מהדפוס הידני להדפסה במכבש, או מהמכונת קיטור למנוע החשמל. אלא שהפעם, המהפכה אינה בתחום החומר הפיזי, אלא בתחום הרוח — בתחום החשיבה, ההבנה והפעולה האינטליגנטית.

במשך דורות, חלמו מדענים ופילוסופים על מכונות חושבות. אלן טיורינג שאל את השאלה המפורסמת: "האם מכונות יכולות לחשוב?" בשנות החמישים והשישים של המאה העשרים, חוקרי בינה מלאכותית בנו מערכות מומחים שניסו לחקות את תהליך קבלת ההחלטות האנושי. אך כל אותן מערכות סבלו ממגבלה יסודית: הן היו כבולות לתחום ידע צר, ויכלו לפעול רק במסגרת חוקים קשיחים שהוגדרו מראש[1].

בעשור האחרון, עם הופעתם של **מודלי שפה גדולים** (LLM), חלה מהפכה אמיתית. לראשונה בהיסטוריה, יש בידינו מערכות המסוגלות להבין שפה טבעית ברמה שנראתה עד לא מזמן כמדע בדיוני.

1.2 המעבר מהבנה לפעולה

תארו לעצמכם מדען מבריק הכלוא בחדר סגור. הוא יכול לחשוב, לנתח, להסיק מסקנות — אך הוא חסר כל יכולת אינטראקציה עם העולם החיצון. זה בדיוק המצב של מודל שפה גדול ללא ממשקי פעולה.

כאן נכנס לתמונה המושג המרכזי של ספר זה: **הסוכן** (Agent). סוכן הוא מערכת המשלבת מודל שפה עם **יכולת פעולה**.

1.3 פרוטוקול הקשר המודל

MCP עושה לעולם הסוכנים האגנטיים בדיוק את מה שסטנדרט החשמל עשה לעולם המכשירים החשמליים: הוא מגדיר **ממשק אחיד** שמאפשר לכל לקוח לתקשר עם כל שרת[2].

Model Context Protocol, שפותח על ידי Anthropic, הוא פרוטוקול פתוח המגדיר כיצד לקוחות יכולים לתקשר עם שרתים.

1.4 למי מיועד הספר

ספר זה מיועד למפתחים ומהנדסי תוכנה המעוניינים להיכנס לעולם החדש של הסוכנים האגנטיים. הוא מניח ידע בסיסי בתכנות, ובפרט היכרות עם Python.

1.5 מבנה הספר

הספר בנוי כמסע מתקדם בארבעה שלבים:

חלק ראשון — יסודות: סקירה של עולם הסוכנים האגנטיים והיכרות עם פרוטוקול MCP.

כל הזכויות שמורות - Dr. Segal Yoram ©

חלק שני — לקוחות MCP: צלילה עמוקה לארכיטקטורת הלקוח, בניית לקוחות, ויכולות מתקדמות.

חלק שלישי — שרתי MCP: הבנת צד השרת, בניית שרתים, והפרימיטיבים המרכזיים.

חלק רביעי — תעבורה ואינטגרציה: שכבות התעבורה השונות ודפוסי אינטגרציה.

1.6 תודות

תודה ל-Anthropic על פיתוח MCP והפיכתו לפרוטוקול פתוח.

חלק I

יסודות

2 מבוא לבינה מלאכותית אגנטית

2.1 פתיחה: מחשבה על המחשבה הפועלת

יש רגע מכונן בהיסטוריה של הפילוסופיה המערבית שבו אריסטו, בספרו De Anima (על הנפש), משרטט את ההבחנה המרכזית בין תיאוריה (theoria) לבין פראקסיס (praxis) — בין ההבנה הפילוסופית הטהורה לבין הפעולה בעולם. אריסטו טען שהידע האמיתי אינו עניין של תבונה בלבד, אלא של יכולת להפעיל את אותה תבונה למען מטרות מעשיות.

כיום, כאשר אנו עומדים בפני התפתחות מרשימה של מודלי שפה גדולים (LLM), אנו מגלים מחדש את אותה דילמה עתיקת יומין. מודל שפה יכול להבין טקסט, לנתח משמעות, אף לייצר תשובות מבריקות — אך כל אלה נשארים בתחום התיאוריה. הקפיצה המהפכנית שאנו חווים כעת היא המעבר מן ההבנה אל הפעולה: מן ה-LLM אל ה**סוכן** (Agent).

ההיסטוריה של המדעים הקוגניטיביים והבינה המלאכותית מלמדת אותנו שהאתגר האמיתי של אינטליגנציה אינו רק בהבנה, אלא ביכולת הפעולה האוטונומית במרחב לא-מוכר. כבר בשנות החמישים, אלן טיורינג העלה את השאלה המפורסמת "האם מכונות יכולות לחשוב?", אך השאלה העמוקה יותר שעולה כיום היא: "האם מכונות יכולות לפעול על סמך חשיבה?"

עם הופעתם של מודלי שפה גדולים, ובמיוחד עם יכולותיהם המתקדמות של דור האחרון (כגון GPT-4, Claude 3, ו-Gemini), אנו עדים לשינוי פרדיגמטי: לראשונה, יש לנו "מוח" טקסטואלי מספיק חזק שיכול לא רק להבין הוראות מורכבות, אלא גם לתכנן רצפי פעולות, לקבל החלטות תוך כדי תנועה, ולהסתגל לסיטואציות חדשות.

2.2 הגדרה פורמלית של סוכן

כדי לדון בסוכנים בצורה מדויקת, עלינו להגדירם באופן פורמלי. בתחום הבינה המלאכותית, סוכן מוגדר כישות המקיימת אינטראקציה מתמשכת עם סביבתה על מנת להשיג מטרות מסוימות[1].

2.2.1 מודל מתמטי של סוכן

נגדיר סוכן באופן פורמלי כך:

S — מרחב המצבים (State Space): כל המצבים האפשריים שהסביבה יכולה להיות בהם.

A — מרחב הפעולות (Action Space): כל הפעולות שהסוכן יכול לבצע.

O — מרחב התצפיות (Observation Space): המידע שהסוכן קולט מהסביבה.

$\pi : S \times O \to A$ — מדיניות (Policy): פונקציה ממצב ותצפית לפעולה.

$T : S \times A \to S$ — פונקציית מעבר (Transition Function): איך המצב משתנה בעקבות פעולה.

$R : S \times A \to \mathbb{R}$ — פונקציית תגמול (Reward Function): הערכת איכות הפעולה.

המטרה של הסוכן היא למצוא מדיניות אופטימלית π^* שממקסמת את התגמול המצטבר

כל הזכויות שמורות - Dr. Segal Yoram ©

לאורך זמן:

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid \pi\right] \tag{1}$$

כאשר $\gamma \in [0,1]$ הוא גורם היוון (Discount Factor), $s_t \in S$ הוא המצב בזמן t, ו-$a_t = \pi(s_t)$ היא הפעולה שנבחרה.

2.2.2 מחזור הפעולה הסוכני

ליבת הפעולה של כל סוכן היא לולאה אינסופית של תצפית-חשיבה-פעולה. מתמטית, ניתן לתאר את הלולאה כרצף של מעברי מצב:

$$S_0 \xrightarrow{a_0} S_1 \xrightarrow{a_1} S_2 \xrightarrow{a_2} \cdots \xrightarrow{a_{t-1}} S_t \xrightarrow{a_t} S_{t+1} \tag{2}$$

בכל צעד t, הסוכן קולט תצפית o_t על המצב הנוכחי s_t, מחשב פעולה $a_t = \pi(s_t, o_t)$ לפי המדיניות שלו, מבצע את הפעולה בסביבה, הסביבה עוברת למצב חדש $s_{t+1} = T(s_t, a_t)$, ומקבל תגמול $r_t = R(s_t, a_t)$.

המודל הזה, הידוע גם כ-**תהליך החלטה של מרקוב** (Markov Decision Process, MDP), הוא הבסיס התיאורטי לכל אלגוריתמי למידת חיזוק ולרבות מאלגוריתמי הסוכנים המודרניים[3].

2.2.3 סוכנים מבוססי LLM

כאשר המוח של הסוכן הוא מודל שפה גדול, המדיניות π מוחלפת בתהליך מורכב יותר של הקידוד (Encoding), הסקה (Inference), ופענוח (Decoding).

נסמן את תהליך זה כ-π_{LLM}:

$$\pi_{\text{LLM}}(s_t, o_t) = \text{decode}\left(\text{LLM}\left(\text{encode}(s_t, o_t)\right)\right) \tag{3}$$

היתרון המרכזי של גישה זו הוא ש-LLM מודרני כבר למד מכמויות אדירות של טקסט, ובכך רכש "ידע עולם" (world knowledge) רחב. זה מאפשר לסוכן לטפל במצבים מורכבים ולא-צפויים ללא צורך באימון ספציפי לכל תחום[4].

2.3 זרימות עבודה אגנטיות לעומת סוכנים אוטונומיים

הבחנה מכרעת בעולם הבינה המלאכותית המעשית היא זו שבין **זרימות עבודה אגנטיות** (Agentic Workflows) לבין **סוכנים אוטונומיים** (Autonomous Agents). שתי הגישות משתמשות במודלי שפה גדולים, אך הן שונות זו מזו באופן מהותי בשאלה של מיקוס השליטה (Locus of Control).

2.3.1 זרימת עבודה אגנטית

זרימת עבודה אגנטית היא רצף של צעדים מתוזמר מראש, שבו ה-LLM משמש כרכיב חישובי בתוך תהליך דטרמיניסטי. המפתח (או האדריכל של המערכת) מגדיר את סדר הפעולות,

והמודל מבצע משימות ספציפיות בכל שלב.

מבנה פורמלי: זרימת עבודה אגנטית היא פונקציה מורכבת קבועה מראש:

$$\text{Workflow}(x) = f_n \circ \text{LLM}_n \circ f_{n-1} \circ \cdots \circ f_1 \circ \text{LLM}_1(x) \tag{4}$$

כאשר x הוא הקלט ההתחלתי, LLM_i הן קריאות למודל השפה, ו-f_i הן פעולות קבועות מראש.

2.3.2 סוכן אוטונומי מלא

סוכן אוטונומי הוא מערכת שבה ה-LLM עצמו מחליט באופן דינמי אילו פעולות לבצע, באיזה סדר, וכמה זמן להמשיך. לסוכן יש מטרה (goal), ויש לו גישה לכלים (tools), אך המסלול המדויק אינו קבוע מראש.

מבנה פורמלי: הסוכן הוא תהליך רקורסיבי:

$$\text{Agent}(g, s_t) = \begin{cases} \text{output} & \text{if LLM decides goal } g \text{ is achieved} \\ \text{Agent}(g, s_{t+1}) & \text{otherwise} \end{cases} \tag{5}$$

שתי הגישות הן כלים חשובים בארגז הכלים של המפתח. הבחירה ביניהן תלויה באופי המשימה[5].

2.4 ארכיטקטורת הליבה של סוכן LLM

כל סוכן מבוסס LLM נבנה משלושה רכיבים מרכזיים, כאשר כל אחד מהם ממלא תפקיד קריטי בתפקוד הכולל[6], [7].

2.4.1 רכיב 1: מודל השפה — המוח החושב

מודל השפה הגדול (LLM) הוא "המוח" של הסוכן. הוא אחראי על הבנת המטרה, תכנון, קבלת החלטות, הסקה, וסינתזה.

המודלים המודרניים מציעים יכולות מרשימות: הבנת הקשר ארוך (Long Context Understanding), חשיבה שרשרתית (Chain-of-Thought, CoT)[8], וקריאת פונקציות מובנית (Function Calling).

2.4.2 רכיב 2: כלים — הידיים הפועלות

כלים (Tools) הם הממשק שבאמצעותו הסוכן פועל בעולם. ללא כלים, ה-LLM נשאר רק "מחשבון טקסט".

טיפולוגיה של כלים: כלי קריאה (Read Tools), כלי כתיבה (Write Tools), כלי חישוב (Computation Tools), וכלי אינטראקציה (Interaction Tools).

המחקר על שימוש בכלים על ידי LLM התפתח מהר בשנים האחרונות[9], [10].

כל הזכויות שמורות - Dr. Segal Yoram ©

2.4.3 רכיב 3: לולאת הסוכן — הלב הפועם

לולאת הסוכן (Agent Loop) היא המנגנון המתאם את פעילות המודל והכלים. היא "הלב הפועם" של הסוכן — מחזור אינסופי של תצפית, חשיבה, פעולה.

ריאקט (tcAeR) — פרדיגמת הפעולה המובילה:

אחת הפרדיגמות המובילות בתכנון סוכנים היא **tcAeR** (ראשי תיבות של Reasoning and Acting), שהוצעה על ידי Yao et al.[11]. הרעיון המרכזי הוא שה-LLM לא רק מחליט על פעולות, אלא גם מנמק את החלטותיו בצורה מפורשת.

בכל צעד, המודל מייצר: **thguohT** (מחשבה), **noitcA** (פעולה), ו-**noitavresbO** (תצפית). המחזור חוזר עד שהמודל מייצר **rewsnA** (תשובה).

2.5 האתגר המרכזי: חיבור סוכנים לעולם

לאחר שהבנו את המבנה הפנימי של סוכנים, עלינו להתמודד עם שאלה מעשית קריטית: כיצד נחבר את הסוכן לעולם החיצוני?

2.5.1 הבעיה: פרגמנטציה ואי-תאימות

עד לשנת 4202, כל מערכת סוכן פיתחה את הממשק שלה לכלים. לכל LLM ולכל פלטפורמה היה פורמט שונה להגדרת כלים, לקריאתם, ולהחזרת תוצאות.

2.5.2 הפתרון: פרוטוקול סטנדרטי

מה שעולם הסוכנים היה צריך זה **פרוטוקול הקשר המודל** (Model Context Protocol, או בקיצור MCP).

MCP הוא פרוטוקול פתוח ומוגדר היטב שמגדיר איך לקוחות (אפליקציות המארחות סוכנים) מתקשרים עם שרתים (שירותים החושפים כלים).

יתרונות MCP: אוניברסליות, יכולת החלפה, אקוסיסטם, ותחזוקה.

MCP פותח על ידי Anthropic[2] והושק בנובמבר 4202 כפרוטוקול פתוח. הוא מבוסס על JSON-RPC[12].

2.6 סיכום הפרק והצעד הבא

בפרק זה פתחנו את הדיון על סוכני בינה מלאכותית מנקודת מבט פילוסופית, היסטורית, ומתמטית.

מה למדנו: הגדרת סוכן כישות אוטונומית; המודל המתמטי כתהליך החלטה; לולאת הסוכן כמחזור של תצפית-חשיבה-פעולה; ההבחנה בין זרימות עבודה לסוכנים אוטונומיים; שלושת הרכיבים: מודל שפה, כלים, ולולאת סוכן; והצורך בפרוטוקול אחיד (MCP).

מה הלאה? בפרק הבא, נצלול לעומק פרוטוקול MCP עצמו.

3 פרוטוקול הקשר המודל — סקירה מקיפה

3.1 על פרוטוקולים וסטנדרטים בהיסטוריה

לפני שנצלול לפרטים הטכניים של Model Context Protocol, נעצור לרגע ונחשוב על משמעותם העמוקה של פרוטוקולים וסטנדרטים בהיסטוריית הטכנולוגיה והתרבות האנושית.

כאשר גוטנברג המציא את דפוס האותיות הנעות במאה ה-51, הוא לא רק המציא מכונה להדפסת ספרים. הוא יצר **סטנדרט** — אותיות בגודל אחיד, ממוינות בסדר קבוע, שאפשרו שכפול מהיר של טקסטים. הסטנדרט הזה הפך את הידע מנכס נדיר ויקר לסחורה נגישה. בדומה, כאשר אייזק פיטמן פיתח את שיטת הקיצורים שלו למכונות כתיבה במאה ה-91, הוא יצר פרוטוקול שאיפשר לבני אדם ולמכונות לתקשר במהירות חסרת תקדים.

בעידן המודרני, הסטנדרטים הפכו לעמוד השדרה של החברה הטכנולוגית. כאשר Tim Berners-Lee פרסם את מפרט ה-HTTP הראשון בשנת 1991, הוא לא בנה "עוד פרוטוקול תקשורת". הוא יצר שפה משותפת שאיפשרה למחשבים בכל העולם לדבר זה עם זה, והניח את הבסיס לאינטרנט כפי שאנו מכירים אותו כיום. מאוחר יותר, תקן USB שינה את פני התעשייה בכך שהציע ממשק אחיד להתקנים — "חבר פעם אחת, עובד תמיד".

היום, אנו עומדים בפני מהפכה דומה בעולם הבינה המלאכותית. מודלי שפה גדולים הפכו לחלק בלתי נפרד מחיינו, אך כל אחד מהם "מדבר" בשפה שונה מעט. כל ספק מגדיר API משלו, כל פלטפורמה דורשת אינטגרציה ייחודית. זה כאילו היינו חוזרים לימים שבהם כל התקן דרש מתאם משלו, כל מכונה דרשה סוג כבל שונה.

Model Context Protocol[2] הוא התשובה לבעיה זו. זהו פרוטוקול סטנדרטי ופתוח, שמטרתו לסטנדרט את הדרך שבה מודלי שפה מתקשרים עם שרתים חיצוניים ומקורות מידע. במקום לבנות אינטגרציה ייחודית לכל מודל ולכל שירות, אנו בונים **פעם אחת לפי הסטנדרט**, ונהנים מתאימות אוניברסלית.

ניתן לחשוב על MCP כ"USB-C עבור סוכני LLM" — ממשק אחיד המאפשר חיבור אוניברסלי בין מודלים לשרתים, בין סוכנים למשאבים, בין רעיונות למימוש.

3.2 מהי הבעיה ש-MCP פותר

כדי להבין את הערך של MCP, נתחיל בהבנת העולם לפניו.

דמיינו שאתם מפתחים סוכן LLM שאמור לעזור למשתמשים לנהל את הקבצים שלהם, לתשאל מסדי נתונים, ולקרוא מידע מאינטרנט. כיצד הייתם בונים מערכת כזו בעולם שלפני MCP?

1. **אינטגרציה עם מערכת הקבצים:** תכתבו קוד ייעודי שמאפשר למודל לקרוא וכתוב קבצים. הקוד הזה יהיה ספציפי ליישום שלכם, ויצטרך להתאים למבנה ה-API של המודל שבחרתם.

2. **אינטגרציה עם מסד נתונים:** תכתבו שכבת אינטגרציה נוספת, שמתרגמת בין בקשות המודל לשאילתות SQL. גם זו תהיה ייחודית ליישום שלכם.

כל הזכויות שמורות - Dr. Segal Yoram ©

3. **אינטגרציה עם שירותי אינטרנט:** תכתבו עוד שכבה, שמאפשרת למודל לשלוח בקשות HTTP ולקבל תשובות. ושוב, הקוד יהיה ייחודי.

עכשיו דמיינו שאתם רוצים לעבור ממודל אחד למודל אחר — למשל, מ-GPT-4 ל-Claude. מה יקרה? **כל האינטגרציות שבניתם לא יעבדו.** תצטרכו לשכתב את הקוד, לכוונן מחדש, ולהתאים את הממשקים.

או דמיינו שמישהו אחר בנה שרת שמספק גישה לשירות חדש ומעניין — למשל, כלי לניתוח תמונות. אם השרת הזה לא נבנה בדיוק עם הממשק שהסוכן שלכם מצפה לו, תצטרכו לכתוב **שכבת תרגום נוספת.** וכל שרת נוסף שתרצו להוסיף ידרוש תרגום דומה.

זהו מצב של **פיצוץ אינטגרציות.** אם יש לכם n סוכנים ו-m שרתים, אתם עלולים להצטרך לכתוב עד $n \times m$ שכבות אינטגרציה שונות. זה לא סקלבילי, לא תחזוקתי, ולא בר-קיימא.

MCP פותר בעיה זו בצורה אלגנטית: במקום אינטגרציות נקודתיות, יש לנו **פרוטוקול משותף.** כל לקוח MCP "מדבר" באותה שפה, וכל שרת MCP "מבין" אותה. השוואה מספרית:

MCP ללא: $O(n \times m)$ אינטגרציות

MCP עם: $O(n + m)$ אימפלמנטציות של הפרוטוקול

זהו שינוי יסודי בסדר הגודל של המורכבות.

3.3 ארכיטקטורת שלוש השכבות

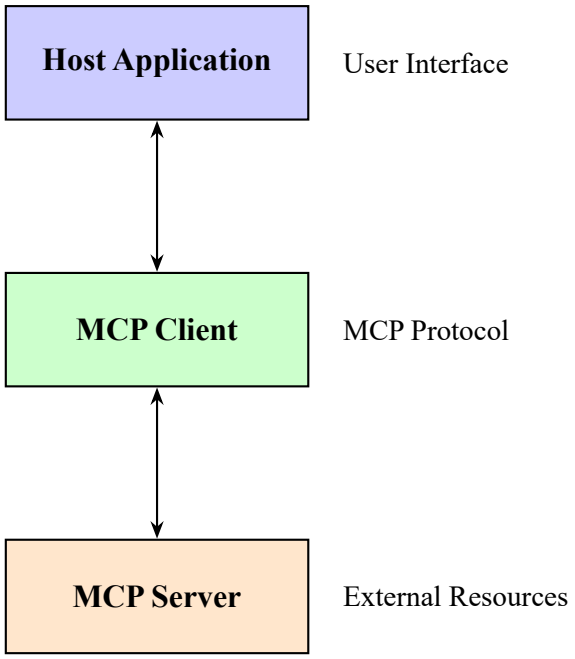
MCP מבוסס על ארכיטקטורה של שלוש שכבות נפרדות ומוגדרות היטב. הפרדה זו משקפת עיקרון עיצובי עמוק: **הפרדת דאגות** (Separation of Concerns).

3.3.1 יישום מארח — Host Application

יישום מארח (Host Application) הוא התוכנה שהמשתמש מפעיל ישירות. זהו הממשק בין האדם למערכת. היישום המארח אחראי על:

- **ממשק משתמש:** הצגת מידע למשתמש וקבלת קלט ממנו
- **ניהול הפעלה:** ניהול מחזור החיים של הסוכן והשרתים
- **אבטחה והרשאות:** קביעת מה מותר ומה אסור לסוכן לעשות
- **תיאום:** תיווך בין המודל (LLM) לשרתי MCP השונים

יישומי מארח נפוצים כוללים:



איור 1: ארכיטקטורת שלוש השכבות של MCP

- **עורכי קוד חכמים:** VS Code, Cursor, Windsurf — סביבות פיתוח המשלבות יכולות LLM עם גישה לקוד
- **יישומי צ'אט:** Claude Desktop — ממשקי שיחה עם המודל
- **כלי שורת פקודה:** Claude Code — ממשקים טרמינל-desab
- **יישומים מותאמים אישית:** כל תוכנה שתבנו עצמכם

3.3.2 לקוח MCP — MCP Client

לקוח MCP (MCP Client) הוא הרכיב **בתוך** היישום המארח שמטפל בתקשורת עם שרתי MCP. מערכת יחסים חשובה: **לכל חיבור לשרת יש לקוח ייעודי** — יחס של אחד לאחד.
אחריות הלקוח:

1. **יצירת חיבור:** פתיחת התקשורת עם השרת וביצוע "לחיצת יד" ראשונית (handshake)
2. **ניהול הפרוטוקול:** שליחת בקשות וקבלת תשובות בפורמט MCP המדויק
3. **תרגום:** המרה בין הפורמט של המודל לפורמט של הפרוטוקול
4. **ניהול מצב:** מעקב אחרי מצב החיבור לאורך זמן
5. **טיפול בשגיאות:** התמודדות עם כשלים, ניתוקים, ו-stuoemit

כל הזכויות שמורות - Dr. Segal Yoram ©

3.3.3 שרת MCP — MCP Server

שרת MCP (MCP Server) הוא תהליך נפרד החושף יכולות דרך הפרוטוקול. השרת הוא ה"שער" למשאבים ולפעולות בעולם החיצוני. הוא מספק שלושה סוגי פרימיטיבים:

1. **כלים** (Tools) — פונקציות שהמודל יכול להפעיל ("עשה משהו")
2. **משאבים** (Resources) — מידע שהמודל יכול לקרוא ("קרא משהו")
3. **הנחיות** (Prompts) — תבניות מוכנות להנחיית המודל ("התנהג כך")

נסמן את קבוצת הפרימיטיבים של שרת S כך:

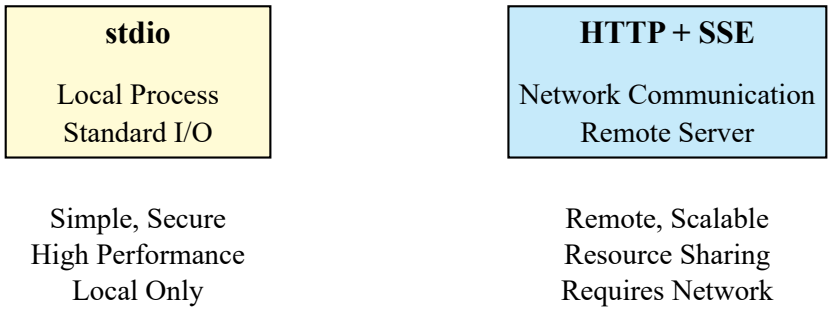
$$S = \{T, R, P\}$$

כאשר:

- $T = \{t_1, t_2, \ldots, t_n\}$ — קבוצת הכלים
- $R = \{r_1, r_2, \ldots, r_m\}$ — קבוצת המשאבים
- $P = \{p_1, p_2, \ldots, p_k\}$ — קבוצת ההנחיות

3.4 שכבות התעבורה — Transport Layers

MCP תומך בשתי שכבות תעבורה עיקריות, כל אחת מתאימה לתרחישי שימוש שונים. בחירת שכבת התעבורה משפיעה על הביצועים, האבטחה, והפריסה של המערכת.



איור 2: שתי שכבות התעבורה העיקריות של MCP

3.4.1 תעבורת קלט/פלט סטנדרטי — stdio Transport

תעבורת stdio[13] (Standard I/O Transport) משתמשת בזרמי הקלט/פלט הסטנדרטיים של תהליכי מערכת ההפעלה. היישום המארח מריץ את השרת כתהליך בן (child process) ומתקשר איתו דרך צינורות (pipes).

מודל התקשורת:

$$\text{Host} \xrightarrow{\text{stdin}} \text{Server} \xrightarrow{\text{stdout}} \text{Host}$$

יתרונות:

- **פשטות:** אין צורך לנהל פורטים, כתובות PI, או חיבורי רשת
- **אבטחה:** התקשורת מוגבלת לתהליך המקומי — אין חשיפה לרשת
- **אמינות:** אין תלות ברשת, בשרתים מרוחקים, או בתנאי תקשורת
- **ביצועים:** השהייה (latency) מינימלית ללא עלות רשת
- **בידוד:** כל לקוח מקבל מופע ייעודי של השרת

חסרונות:

- **מקומיות בלבד:** השרת חייב לרוץ על אותה מכונה כמו הלקוח
- **חוסר שיתוף:** כל לקוח צריך להריץ שרת משלו
- **תלות בפלטפורמה:** שרת שנבנה ל-Windows לא יעבוד על Linux וההפך

3.4.2 תעבורת HTTP זורמת — Streamable HTTP Transport

תעבורת HTTP זורמת[14] (Streamable HTTP Transport) מאפשרת תקשורת דרך הרשת באמצעות פרוטוקולי אינטרנט סטנדרטיים. תעבורה זו משלבת:

1. **בקשות HTTP רגילות** — לפעולות נקודתיות (esnopser-tseuqer)
2. **SSE** (Server-Sent Events) — לעדכונים בזמן אמת מהשרת ללקוח

מודל התקשורת:

$$\text{Client} \xrightarrow{\text{HTTP POST}} \text{Server} \xrightarrow{\text{HTTP Response}} \text{Client}$$

$$\text{Server} \xrightarrow{\text{SSE Stream}} \text{Client}$$

יתרונות:

- **שרתים מרוחקים:** השרת יכול לרוץ בכל מקום בעולם
- **שיתוף משאבים:** מספר לקוחות יכולים להתחבר לאותו שרת
- **סקלביליות:** קל להרחיב למספר רב של משתמשים

כל הזכויות שמורות - Dr. Segal Yoram ©

- **ניהול מרכזי:** עדכון השרת פעם אחת משפיע על כל הלקוחות
- **גמישות פריסה:** שרתים ב-duolc, egde, או sretnec atad

חסרונות:

- **מורכבות:** דרישה לניהול אבטחה, אימות, והרשאות
- **תלות ברשת:** ביצועים תלויים באיכות החיבור
- **עלות:** שרתים מרוחקים דורשים תשתית והוצאות תפעול

3.4.3 השוואת שכבות התעבורה

טבלה 1: השוואה בין שכבות התעבורה של MCP

קריטריון	stdio	HTTP/SSE
פשטות הגדרה	✓✓✓	✓
אבטחה מובנית	✓✓✓	✓
ביצועים	✓✓✓	✓✓
שרתים מרוחקים	□	✓✓✓
שיתוף בין לקוחות	□	✓✓✓
עלות תשתית	✓✓✓	✓
סקלביליות	✓	✓✓✓

3.5 פרוטוקול התקשורת — JSON-RPC

MCP מבוסס על JSON-RPC 0.2[12] — פרוטוקול פשוט, מוכח, ונפוץ להעברת קריאות פונקציה מרחוק (Remote Procedure Call — RPC) בפורמט JSON.

3.5.1 מדוע JSON-RPC?

הבחירה ב-JSON-RPC איננה מקרית. פרוטוקול זה מציע מספר יתרונות מהותיים:

1. **פשטות:** מבנה הודעות ברור וקל להבנה
2. **נפוצות:** ספריות לקוח ושרת זמינות בכל שפות התכנות
3. **עצמאות שפה:** JSON נתמך בכל פלטפורמה מודרנית
4. **קריאות:** הודעות ניתנות לקריאה אנושית, מקל על ניפוי באגים
5. **מבנה:** הבחנה ברורה בין בקשות, תשובות, והתראות

3.5.2 מבנה ההודעות

הפרוטוקול מגדיר שלושה סוגי הודעות בסיסיים. נסמן:

- Req — קבוצת הבקשות (Requests)
- Res — קבוצת התשובות (Responses)
- Notif — קבוצת ההתראות (Notifications)

כל הודעה m במערכת שייכת לאחת מהקבוצות:

$$m \in \text{Req} \cup \text{Res} \cup \text{Notif}$$

1. בקשות (Requests) — הודעות הדורשות תשובה מהצד השני:

מבנה בקשת JSON-RPC

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_file",
        "arguments": {
            "path": "/home/user/data.txt"
        }
    }
}
```

מבנה פורמלי:

$$\text{Req} = \{\text{jsonrpc} : \text{“2.0”}, \text{id} : \mathbb{Z} \cup \text{String}, \text{method} : \text{String}, \text{params} : \text{Object}\}$$

2. תשובות (Responses) — תגובות לבקשות, כוללות תוצאה או שגיאה:

כל הזכויות שמורות - Dr. Segal Yoram ©

מבנה תשובת JSON-RPC — הצלחה

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "result": {
        "content": [
            {
                "type": "text",
                "text": "File␣contents␣here..."
            }
        ]
    }
}
```

מבנה תשובת JSON-RPC — שגיאה

```
{
    "jsonrpc": "2.0",
    "id": 1,
    "error": {
        "code": -32600,
        "message": "Invalid␣request"
    }
}
```

מבנה פורמלי:

$$\text{Res} = \{\text{jsonrpc} : \text{“2.0”}, \text{id} : \mathbb{Z} \cup \text{String}, (\text{result} \mid \text{error})\}$$

כאשר:

$$\text{result} \in \text{Any} \quad \vee \quad \text{error} = \{\text{code} : \mathbb{Z}, \text{message} : \text{String}\}$$

3. התראות (Notifications) — הודעות חד-כיווניות ללא תשובה:

מבנה התראת JSON-RPC

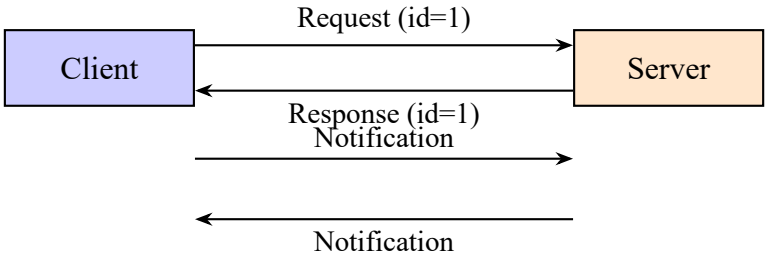
```
{
    "jsonrpc": "2.0",
    "method": "notifications/progress",
    "params": {
        "progress": 50,
        "total": 100
    }
}
```

מבנה פורמלי:

$$\text{Notif} = \{\text{jsonrpc} : \text{“2.0”}, \text{method} : \text{String}, \text{params} : \text{Object}\}$$

שימו לב: התראות **אינן מכילות** שדה `di` כי אין ציפייה לתשובה.

3.5.3 זרימת הודעות דו-כיוונית



איור 3: זרימת הודעות JSON-RPC בין לקוח לשרת

3.6 מחזור חיי החיבור — Connection Lifecycle

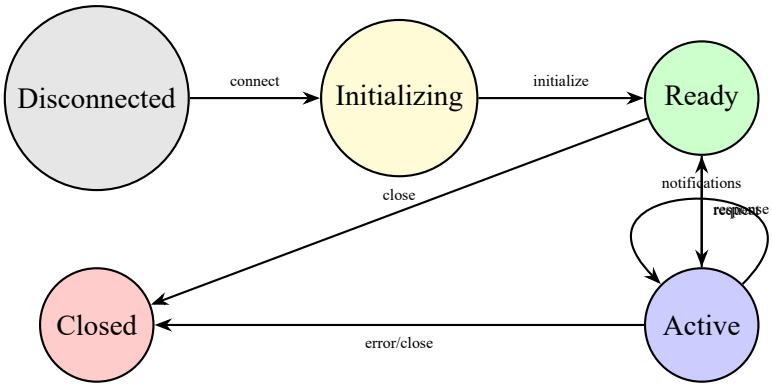
חיבור MCP עובר מספר שלבים מוגדרים היטב, המהווים מכונת מצבים (state machine). הבנת המעברים בין המצבים חיונית לבניית לקוחות ושרתים יציבים.

3.6.1 מכונת המצבים של החיבור

נסמן את מצבי החיבור:

$$\begin{aligned} S &= \{s_0, s_1, s_2, s_3, s_4\} \\ &= \{\text{Disconnected}, \text{Initializing}, \text{Ready}, \text{Active}, \text{Closed}\} \end{aligned}$$

כל הזכויות שמורות - Dr. Segal Yoram ©



איור 4: מכונת מצבים של חיבור MCP

3.6.2 שלב האתחול — Initialization Phase

בשלב האתחול, הלקוח והשרת מחליפים מידע על היכולות (capabilities) שלהם. זהו "לחיצת היד" (handshake) שמבטיחה תאימות.

רצף האתחול:

בקשת אתחול

```
2.0",

nitialize",

lVersion": "2024-11-05",
ities": {
ts": {
"listChanged": true


nfo": {
e": "ExampleClient",
sion": "1.0.0"
```

1. **הלקוח שולח בקשת `ezilaitini`:**

תשובת אתחול

```
oc": "2.0",
,
t": {
rotocolVersion": "2024-11-05",
apabilities": {
  "tools": {},
  "resources": {
      "subscribe": true
  }

erverInfo": {
  "name": "ExampleServer",
  "version": "1.0.0"
```

2. **השרת מגיב עם היכולות שלו:**

התראת השלמת אתחול

```
0",
ifications/initialized"
```

3. **הלקוח שולח התראת `dezilaitini`:**

4. **החיבור מוכן לשימוש** — מעבר למצב `ydaeR`

במונחים פורמליים:

$$s_0 \xrightarrow{\text{connect}} s_1 \xrightarrow{\text{initialize+initialized}} s_2$$

3.6.3 שלב הפעולה — Active Phase

בשלב הפעולה, הלקוח יכול לבצע מגוון פעולות. המעבר ממצב `ydaeR` ל-`evitcA` מתרחש עם כל בקשה:

- **גילוי פרימיטיבים:**
  - `tsil/sloot` — קבלת רשימת הכלים הזמינים
  - `tsil/secruoser` — קבלת רשימת המשאבים
  - `tsil/stpmorp` — קבלת רשימת ההנחיות

כל הזכויות שמורות - Dr. Segal Yoram ©

- **הפעלת כלים:**

  - `llac/sloot` — ביצוע כלי עם פרמטרים

- **קריאת משאבים:**

  - `daer/secruoser` — קריאת תוכן משאב

- **הרשמה לעדכונים:**

  - `ebircsbus/secruoser` — הרשמה לשינויים במשאב
  - `ebircsbusnu/secruoser` — ביטול הרשמה

פונקציית המעבר בין מצבים:

$$\delta : S \times \Sigma \to S$$

כאשר Σ היא קבוצת האירועים (בקשות, תשובות, שגיאות).

3.6.4 שלב הסיום — Termination Phase

בשלב הסיום, החיבור נסגר בצורה מסודרת (graceful shutdown):

1. אחד הצדדים מחליט לסיים את החיבור
2. שליחת הודעת סיום או סגירת ערוץ התקשורת
3. הצד השני מאשר ומשחרר משאבים
4. מעבר למצב `desolC`

מצב סופי:

$$s_4 = \text{Closed} \quad \Rightarrow \quad \forall \sigma \in \Sigma : \delta(s_4, \sigma) = s_4$$

3.7 הפרימיטיבים של MCP

MCP מגדיר שלושה סוגי פרימיטיבים המהווים את "אבני הבניין" של כל שרת. הבנת ההבדלים ביניהם חיונית.

3.7.1 כלים — פונקציות לביצוע

כלים (Tools) הם פונקציות שהמודל יכול להפעיל. הם מייצגים את יכולת ה"עשייה" של הסוכן — פעולות שמשנות מצב או מבצעות חישובים.

כל כלי $t \in T$ מוגדר על ידי:

$$t = (\text{name}, \text{description}, \text{schema}_{\text{input}})$$

- $\text{name} \in \text{String}$ — שם ייחודי המזהה את הכלי
- $\text{description} \in \text{String}$ — תיאור המסביר למודל מתי להשתמש
- $\text{schema}_{\text{input}}$ — סכמת JSON Schema המגדירה את הקלט הצפוי

דוגמה לכלי:

הגדרת כלי לקריאת קובץ

```
{
    "name": "read_file",
    "description": "Read␣the␣contents␣of␣a␣file␣from␣the␣filesystem",
    "inputSchema": {
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": "Path␣to␣the␣file␣to␣read"
            }
        },
        "required": ["path"]
    }
}
```

3.7.2 משאבים — מידע לקריאה

משאבים (Resources) הם מקורות מידע שהמודל יכול לקרוא. הם מייצגים את יכולת ה"ידיעה" של הסוכן — גישה למידע ללא שינויו.

כל משאב $r \in R$ מוגדר על ידי:

$$r = (\text{uri}, \text{name}, \text{description}, \text{mimeType})$$

- $\text{uri} \in \text{URI}$ — מזהה ייחודי בפורמט URI
- $\text{name} \in \text{String}$ — שם קריא לאדם
- $\text{description} \in \text{String}$ — תיאור תוכן המשאב

כל הזכויות שמורות - © Dr. Segal Yoram

- mimeType ∈ String — סוג MIME (לדוגמה, nialp/txet)

דוגמה למשאב:

הגדרת משאב

```
"uri": "file:///home/user/docs/report.txt",
"name": "Monthly␣Report",
"description": "The␣monthly␣sales␣report␣for␣Q4␣2024",
"mimeType": "text/plain"
```

3.7.3 הנחיות — תבניות מוכנות

הנחיות (Prompts) הן תבניות מוכנות שמנחות את המודל לבצע משימות ספציפיות. הן מייצגות "דרכי עבודה" מוכנות שהמשתמש יכול להפעיל.

כל הנחיה $p \in P$ מוגדרת על ידי:

$$p = (\text{name}, \text{description}, \text{arguments})$$

- name ∈ String — שם ייחודי
- description ∈ String — תיאור המטרה
- arguments ⊆ String × Type — רשימת פרמטרים

דוגמה להנחיה:

הגדרת הנחיה לסקירת קוד

```
"name": "code_review",
"description": "Perform␣a␣comprehensive␣code␣review",
"arguments": [
    {
        "name": "file_path",
        "description": "Path␣to␣the␣code␣file␣to␣review",
        "required": true
    },
    {
        "name": "focus",
        "description": "Aspect␣to␣focus␣on␣(security,␣performance,␣
yle)",
        "required": false
    }
]
```

3.8 סיכום הפרק

בפרק זה סקרנו את יסודות Model Context Protocol:

1. **הפילוסופיה:** ראינו כיצד פרוטוקולים וסטנדרטים שינו את פני ההיסטוריה, ואיך MCP ממשיך מסורת זו בעולם הבינה המלאכותית.
2. **הבעיה:** הבנו את בעיית הפיצוץ האינטגרטיבי ואת הצורך בפרוטוקול משותף.
3. **הארכיטקטורה:** למדנו על שלוש השכבות — מארח, לקוח, ושרת — והתפקיד של כל אחת.
4. **שכבות התעבורה:** השווינו בין stdio ו-SSE/HTTP והבנו מתי להשתמש בכל אחד.
5. **הפרוטוקול:** התעמקנו ב-JSON-RPC 0.2 והבנו את מבנה ההודעות.
6. **מחזור החיים:** למדנו על מכונת המצבים של חיבור MCP — אתחול, פעולה, וסיום.
7. **הפרימיטיבים:** הכרנו את שלושת אבני הבניין — כלים, משאבים, והנחיות.

בפרק הבא נתמקד בצד הלקוח ונלמד כיצד לבנות לקוחות MCP מאפס. נצלול לעומק ארכיטקטורת הלקוח, נבין כיצד לנהל חיבורים, ונראה דוגמאות קוד מעשיות.

המפתח להבנת MCP הוא לזכור: **זהו פרוטוקול פשוט ואלגנטי** שפותר בעיה מורכבת. הפשטות היא עוצמתו.

חלק II

לקוחות MCP

כל הזכויות שמורות - Dr. Segal Yoram ©

4 ארכיטקטורת לקוח MCP

4.1 מודל לקוח-שרת בהיסטוריה של המחשוב

כאשר אנו מדברים על ארכיטקטורת לקוח-שרת, אנו נוגעים באחד מהמושגים המכוננים ביותר של מדעי המחשב. מודל זה, שהתפתח במשך עשרות שנים, מייצג תובנה עמוקה על אופן הפצת אינטליגנציה ויכולות במערכות מבוזרות.

אם נסתכל לאחור בהיסטוריה של המחשוב, נגלה שהדיאלקטיקה בין ריכוזיות לפיזור הייתה מתמדת. בשנות השישים של המאה העשרים, מיינפריימים (mainframes) ריכזו את כל הכוח החישובי במרכז אחד, ותחנות עבודה "טיפשות" (dumb terminals) שימשו רק לקלט ולפלט. בשנות השמונים, המחשב האישי הביא לביזור מוחלט — כל מחשב היה עצמאי לחלוטין. אך בשנות התשעים, עם הופעת האינטרנט, שבנו לאיזון חדש: מודל הלקוח-שרת[15].

מודל זה מכיר בעובדה שיש **אסימטריה טבעית** בין שני צדדים של כל תקשורת. צד אחד — השרת — מחזיק במשאב או יכולת. הצד השני — הלקוח — רוצה לגשת למשאב זה. השרת הוא מארח (host), הלקוח הוא אורח (guest). השרת פסיבי ומחכה לבקשות, הלקוח אקטיוי ויוזם אינטראקציות.

בעולם של סוכני LLM עם MCP, מודל הלקוח-שרת מקבל משמעות חדשה ומעמיקה. הלקוח אינו רק יישום שמבקש מידע — הוא הממשק בין אינטליגנציה מלאכותית לבין יכולות קוגניטיביות מורחבות. השרתים אינם רק מאגרי מידע — הם הרחבות של הזיכרון והחושים של הסוכן.

4.2 מודל פורמלי של לקוח MCP

לפני שנצלול לפרטי המימוש, נגדיר מודל פורמלי של לקוח MCP. גישה מתמטית זו תעזור לנו להבין את המבנה העמוק של הארכיטקטורה.

4.2.1 הגדרת מצב הלקוח

לקוח MCP ניתן לתיאור כמערכת בעלת מצב (stateful system), שבה המצב מתפתח לאורך זמן בתגובה לאירועים פנימיים וחיצוניים. נגדיר את מצב הלקוח באופן פורמלי:

$$C = (S, T, M, R) \tag{6}$$

כאשר:

- $S = \{s_1, s_2, \ldots, s_n\}$ הוא קבוצת הסשנים הפעילים, כאשר כל s_i מייצג חיבור לשרת i
- $T = \{t_1, t_2, \ldots, t_m\}$ הוא קבוצת הכלים הזמינים מכל השרתים
- M הוא התור של ההודעות הממתינות לטיפול
- $R = \{r_1, r_2, \ldots, r_k\}$ הוא קבוצת המשאבים המוכרים ללקוח

4.2.2 פונקציות המעבר

מצב הלקוח משתנה דרך פונקציות מעבר. נגדיר שלוש פונקציות עיקריות:

חיבור לשרת:

$$\text{connect} : C \times \text{ServerConfig} \to C' \tag{7}$$

חיבור לשרת חדש מוסיף סשן חדש ומרחיב את קבוצת הכלים והמשאבים:

$$C' = (S \cup \{s_{new}\}, T \cup T_{new}, M, R \cup R_{new}) \tag{8}$$

ביצוע כלי:

$$\text{execute} : C \times \text{ToolName} \times \text{Args} \to C' \times \text{Result} \tag{9}$$

ביצוע כלי משנה את תור ההודעות ומייצר תוצאה.

ניתוק משרת:

$$\text{disconnect} : C \times \text{SessionId} \to C' \tag{10}$$

ניתוק מסיר את הסשן ומסנן כלים ומשאבים שהיו תלויים בו.

4.3 רכיבי הליבה של לקוח MCP

עתה שיש לנו הבנה פורמלית, נבחן את הרכיבים הקונקרטיים המרכיבים לקוח MCP מודרני. כל רכיב אחראי על היבט ספציפי של התפקוד.

4.3.1 מנהל הסשנים

מנהל הסשנים (SessionManager) הוא הרכיב האחראי על מחזור החיים של כל החיבורים. הוא מנהל מיפוי בין מזהי שרתים לאובייקטי סשן, ומבטיח שכל חיבור מטופל כראוי.

אחריות מנהל הסשנים כוללת:

- יצירת סשנים חדשים לפי פרמטרי קונפיגורציה
- ביצוע לחיצת יד ראשונית (handshake) עם כל שרת
- ניטור בריאות הסשנים באמצעות בדיקות heartbeat
- התאוששות אוטומטית מכשלים זמניים
- סגירה מסודרת של חיבורים בעת השבתה

4.3.2 רישום הכלים

רישום הכלים (ToolRegistry) מנהל מאגר מרכזי של כל הכלים הזמינים מכל השרתים. זהו קומפוננט קריטי המאפשר למודל לגלות ולהפעיל כלים ללא ידיעה מוקדמת על מיקומם.

הרישום שומר עבור כל כלי:

כל הזכויות שמורות - Dr. Segal Yoram ©

- שם הכלי (ייחודי גלובלית)
- תיאור פונקציונלי של הכלי
- סכמת הפרמטרים ב-amehcS JSON
- הפניה לסשן המקורי (לאיזה שרת הכלי שייך)
- מטאדאטה נוסף כמו זמן אחזור אחרון

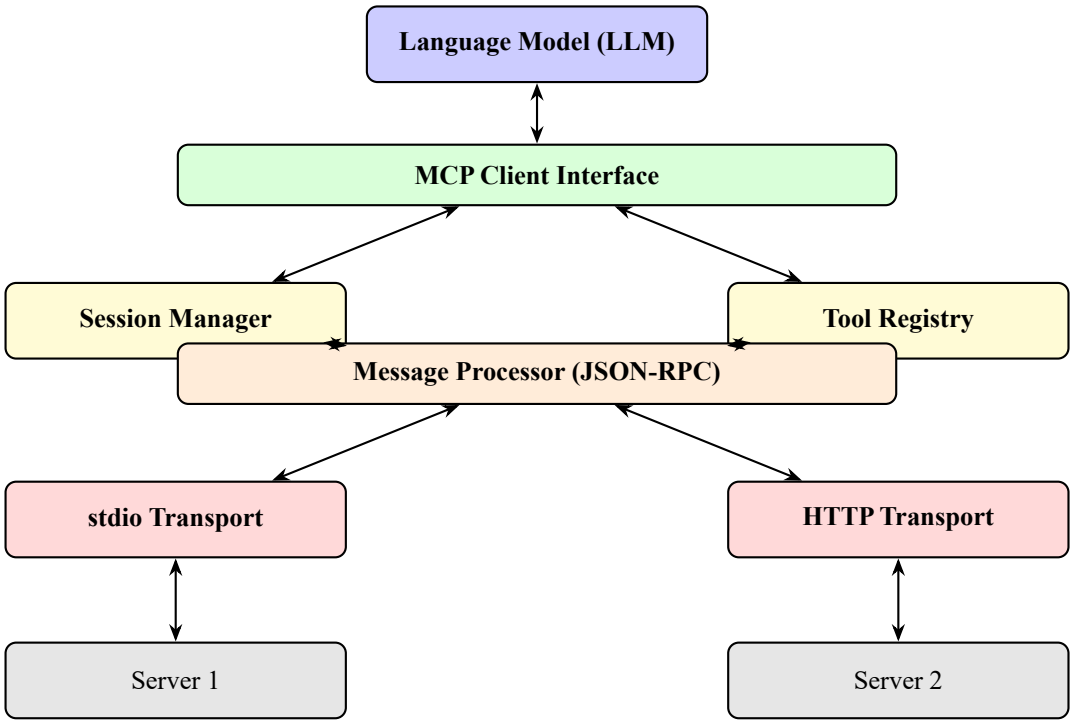
4.3.3 מטפל התעבורה

מטפל התעבורה (TransportHandler) מופשט את שכבת התקשורת הפיזית. הוא מספק ממשק אחיד לשליחת וקבלת הודעות JSON-RPC, בלי קשר לסוג התעבורה הבסיסי — בין אם זה stdio או HTTP.

הפשטה זו מאפשרת למעלות הגבוהות יותר של הלקוח לעבוד בצורה אחידה, ומפשטת את הוספת סוגי תעבורה חדשים בעתיד.

4.4 ארכיטקטורה פנימית של הלקוח

הארכיטקטורה הפנימית של לקוח MCP מורכבת משכבות מוגדרות היטב, כאשר כל שכבה מסתמכת על השכבה שמתחתיה. תרשים הארכיטקטורה הבא ממחיש את הקשרים:



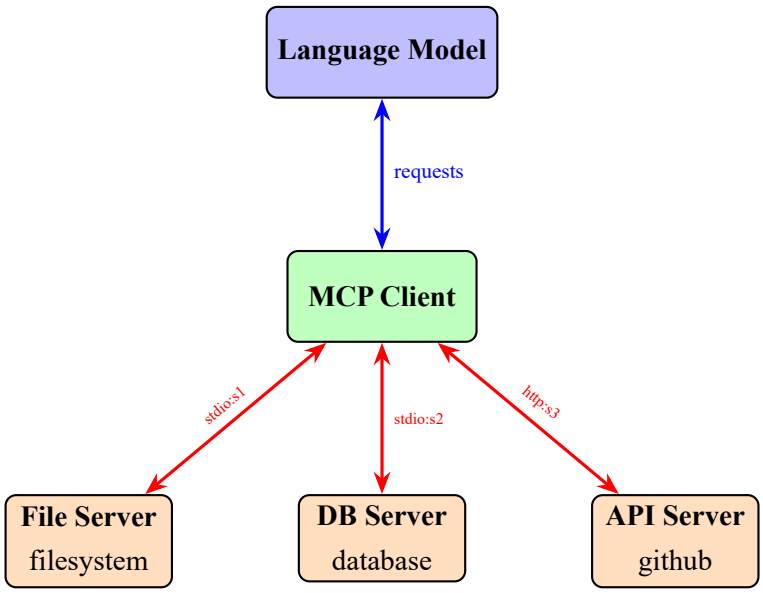
4.5 טופולוגיית חיבור רב-שרתית

אחד היתרונות המרכזיים של MCP הוא היכולת של לקוח להתחבר למספר שרתים במקביל. זה מאפשר לסוכן לגשת למגוון רחב של יכולות מקורות שונים, ולשלב אותן למשימה אחת

קוהרנטית.

4.5.1 מודל חיבור מרובה

בארכיטקטורה של MCP, לקוח אחד יכול לנהל n חיבורים פעילים לשרתים שונים. כל חיבור הוא עצמאי, עם סשן ייחודי, אך הלקוח מאחד את היכולות מכל השרתים לממשק אחיד למודל.



4.5.2 ניהול מרחב שמות

כאשר מספר שרתים מספקים כלים, עלולות להיווצר התנגשויות שמות. לדוגמה, שני שרתים שונים עשויים לספק כלי בשם `daer`. הלקוח צריך לפתור זאת.

ישנן מספר אסטרטגיות:

1. **קידומת שם השרת** — כל כלי מקבל קידומת של שם השרת: `daer.sf`, `daer.bd`
2. **מרחבי שמות מפורשים** — שימוש בסמנטיקה של namespaces
3. **עדיפות לפי סדר** — הכלי הראשון שנמצא "מנצח"
4. **בקשת הבהרה מהמודל** — השארת ההחלטה למודל כאשר יש אי-בהירות

הגישה המומלצת היא קידומת שם השרת, המספקת איזון טוב בין בהירות ופשטות.

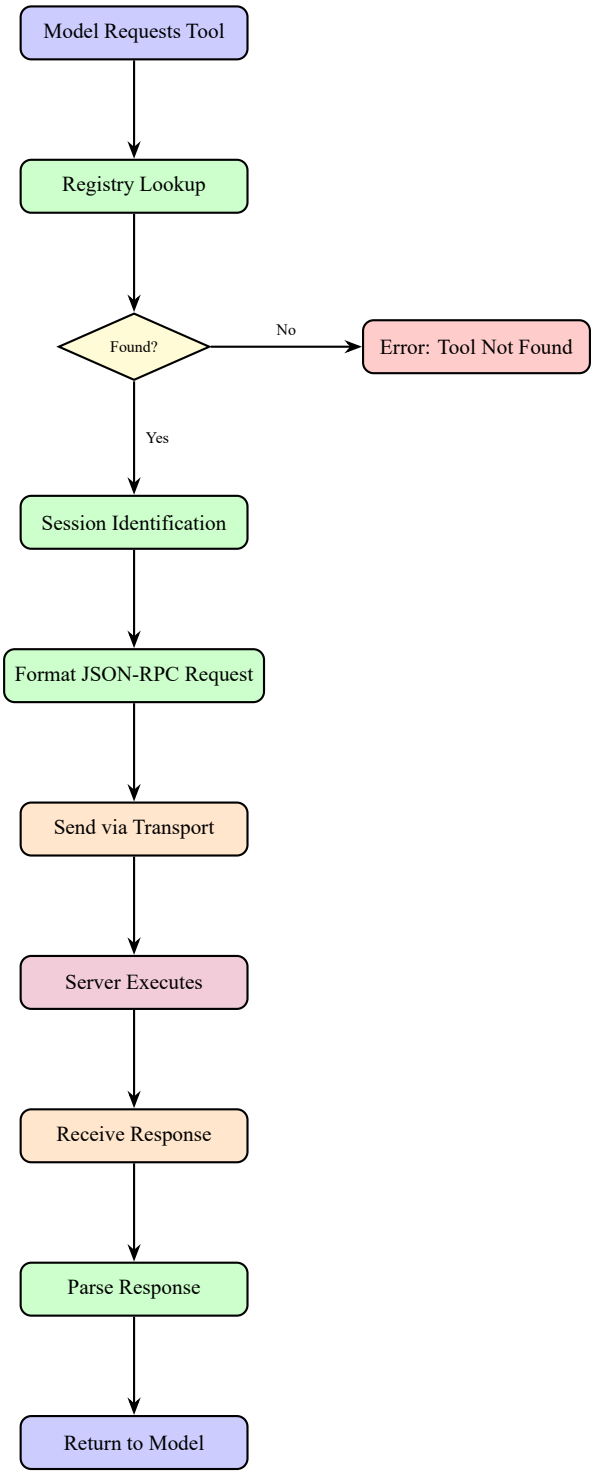
4.6 זרימת הודעות בין לקוח לשרתים

הבנת זרימת ההודעות היא קריטית לעבודה עם MCP. כל אינטראקציה בין הלקוח לשרת עוברת מחזור חיים מוגדר היטב.

כל הזכויות שמורות - © Dr. Segal Yoram

4.6.1 מחזור חיי בקשה-תשובה

תרשים הזרימה הבא מתאר את המסע המלא של בקשת כלי:



4.7 יישום לקוח בסיסי ב-Python

כעת נעבור מתיאוריה למימוש. נראה כיצד בונים לקוח MCP בסיסי באמצעות ה-SDK הרשמי של Python.

4.7.1 יצירת סשן יחיד

הדוגמה הבאה מראה יצירת חיבור לשרת MCP יחיד:

כל הזכויות שמורות - Dr. Segal Yoram ©

יצירת סשן לקוח בסיסי

```python
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from contextlib import AsyncExitStack
import asyncio

async def create_basic_client():
    """Create␣a␣basic␣MCP␣client␣connected␣to␣one␣server."""
    # Create exit stack for resource management
    exit_stack = AsyncExitStack()

    # Define server parameters
    server_params = StdioServerParameters(
        command="python",
        args=["my_mcp_server.py"],
        env={"LOG_LEVEL": "INFO"}
    )

    # Create stdio transport
    read, write = await exit_stack.enter_async_context(
        stdio_client(server_params)
    )

    # Create session
    session = await exit_stack.enter_async_context(
        ClientSession(read, write)
    )

    # Initialize with handshake
    init_result = await session.initialize()
    print(f"Connected␣to:␣{init_result.serverInfo.name}")
    print(f"Version:␣{init_result.serverInfo.version}")

    return session, exit_stack

# Usage
async def main():
    session, stack = await create_basic_client()

    # Use the session...
    tools = await session.list_tools()
    print(f"Available␣tools:␣{[t.name␣for␣t␣in␣tools.tools]}")

    # Cleanup
    await stack.aclose()

asyncio.run(main())
```

4.7.2 מחלקת לקוח רב-שרתי

עבור יישום ייצור, נרצה לקוח שמנהל מספר שרתים. הנה מימוש מלא:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
from typing import Dict, List, Tuple, Optional
from dataclasses import dataclass
import logging

@dataclass
class ServerConfig:
    """Configuration␣for␣a␣single␣MCP␣server."""
    name: str
    command: str
    args: List[str]
    env: Optional[Dict[str, str]] = None

class MultiServerClient:
    """MCP␣client␣managing␣multiple␣server␣connections."""

    def __init__(self):
        self.sessions: Dict[str, ClientSession] = {}
        self.tools: Dict[str, Tuple[str, Tool]] = {}
        self.resources: Dict[str, Tuple[str, Resource]] = {}
        self.exit_stack = AsyncExitStack()
        self.logger = logging.getLogger(__name__)

    async def connect_server(self, config: ServerConfig):
        """Connect␣to␣a␣new␣MCP␣server."""
        try:
            # Create server parameters
            params = StdioServerParameters(
                command=config.command,
                args=config.args,
                env=config.env or {}
            )

            # Create transport
            read, write = await self.exit_stack.enter_async_context(
                stdio_client(params)
            )

            # Create and initialize session
            session = await self.exit_stack.enter_async_context(
                ClientSession(read, write)
            )
            await session.initialize()

            # Store session
            self.sessions[config.name] = session

            # Register tools and resources
            await self._register_capabilities(config.name, session)

            self.logger.info(f"Connected␣to␣server:␣{config.name}")
```

המשך: ביצוע כלים וניהול משאבים

```
    async def call_tool(
        self,
        tool_name: str,
        arguments: dict
    ) -> dict:
        """Call␣a␣tool,␣routing␣to␣the␣correct␣server."""
        if tool_name not in self.tools:
            raise ValueError(f"Unknown␣tool:␣{tool_name}")

        server_name, tool = self.tools[tool_name]
        session = self.sessions[server_name]

        self.logger.debug(f"Calling␣{tool_name}␣on␣{server_name}")

        try:
            result = await session.call_tool(tool.name, arguments)
            return {"success": True, "result": result}
        except Exception as e:
            self.logger.error(f"Tool␣call␣failed:␣{e}")
            return {"success": False, "error": str(e)}

    async def read_resource(self, resource_uri: str) -> dict:
        """Read␣a␣resource␣from␣the␣appropriate␣server."""
        if resource_uri not in self.resources:
            raise ValueError(f"Unknown␣resource:␣{resource_uri}")

        server_name, resource = self.resources[resource_uri]
        session = self.sessions[server_name]

        result = await session.read_resource(resource.uri)
        return result

    def list_all_tools(self) -> List[str]:
        """Get␣list␣of␣all␣available␣tools."""
        return list(self.tools.keys())

    def list_all_resources(self) -> List[str]:
        """Get␣list␣of␣all␣available␣resources."""
        return list(self.resources.keys())

    async def close(self):
        """Close␣all␣connections␣gracefully."""
        self.logger.info("Closing␣all␣server␣connections")
        await self.exit_stack.aclose()
```


© Dr. Segal Yoram - כל הזכויות שמורות

4.8 טיפול בשגיאות ועמידות

לקוח MCP איכותי צריך להיות עמיד בפני כשלים. מערכות מבוזרות נוטות להיכשל, והלקוח צריך להתמודד עם זה בחן.

4.8.1 סוגי שגיאות נפוצות

טבלה 2: סוגי שגיאות נפוצות והטיפול בהן

אסטרטגיית טיפול	תיאור	סוג שגיאה
ניסיון חוזר עם backoff אקספוננציאלי	שרת לא זמין או ניתוק	ConnectionError
הגדרת timeout והודעה ברורה	פעולה ארכה מדי	TimeoutError
רישום השגיאה וכשל מיידי	תשובה לא תקינה	ProtocolError
בדיקה מקומית לפני שליחה	כלי לא קיים	ToolNotFound
ולידציה מול JSON Schema	פרמטרים שגויים	InvalidArguments

4.8.2 מנגנון ניסיונות חוזרים

הנה יישום של מנגנון ניסיונות חוזרים עם exponential backoff:

© Dr. Segal Yoram - כל הזכויות שמורות

טיפול בשגיאות עם ניסיונות חוזרים

```
import asyncio
from typing import Callable, TypeVar, Any

T = TypeVar('T')

async def retry_with_backoff(
    func: Callable[..., T],
    max_retries: int = 3,
    initial_delay: float = 1.0,
    backoff_factor: float = 2.0,
    max_delay: float = 60.0
) -> T:
    """Execute␣function␣with␣exponential␣backoff␣retry."""
    delay = initial_delay

    for attempt in range(max_retries):
        try:
            return await func()
        except (ConnectionError, asyncio.TimeoutError) as e:
            if attempt == max_retries - 1:
                raise

            # Log the retry
            logging.warning(
                f"Attempt␣{attempt␣+␣1}␣failed:␣{e}.␣"
                f"Retrying␣in␣{delay:.2f}s..."
            )

            # Wait before retry
            await asyncio.sleep(delay)

            # Increase delay for next attempt
            delay = min(delay * backoff_factor, max_delay)

async def safe_tool_call(
    client: MultiServerClient,
    tool_name: str,
    args: dict,
    timeout: float = 30.0
) -> dict:
    """Call␣a␣tool␣with␣comprehensive␣error␣handling."""
    try:
        # Validate tool exists
        if tool_name not in client.tools:
            return {
                "success": False,
                "error": f"Tool␣'{tool_name}'␣not␣found"
            }

        # Call with retry and timeout
```


© Dr. Segal Yoram - כל הזכויות שמורות

4.9 חלוקת אחריות: לקוח מול שרת

הבנת החלוקה בין אחריות הלקוח לאחריות השרת היא קריטית לתכנון נכון של מערכת MCP.

טבלה 3: חלוקת אחריות בין לקוח לשרת

אחריות השרת	אחריות הלקוח	רכיב
קבלת חיבורים, לחיצת יד	יצירה, ניהול, סגירה	**חיבור**
חשיפת כלים ומשאבים	שאילת יכולות, בניית רישום	**גילוי**
ביצוע לוגיקה בפועל	ניתוב לשרת נכון, תזמון	**ביצוע**
דיווח ברור על כשלים	ניסיונות חוזרים, פיילאובר	**שגיאות**
ולידציה, הגנה על משאבים	אימות משתמש, הרשאות	**אבטחה**
עדכון על שינויים	שמירת תשובות, אופטימיזציה	**מטמון**

העיקרון המנחה: **הלקוח מתאם, השרת מבצע**. הלקוח אחראי על האורקסטרציה — הוא מחליט מתי לקרוא לאיזה שרת. השרת אחראי על היישום — הוא מבצע את הפעולה בפועל.

4.10 אופטימיזציה וביצועים

לקוח יעיל הוא קריטי לחווית משתמש טובה. ישנן מספר טכניקות לשיפור ביצועים.

4.10.1 מטמון תשובות

כלים מסוימים מחזירים תוצאות שאינן משתנות לעיתים קרובות. לדוגמה, קריאת קובץ תצורה או שאילת נתונים היסטוריים. מטמון חכם יכול להפחית עומס ולהאיץ תגובות.

מטמון פשוט לתשובות כלים

```python
from datetime import datetime, timedelta
from typing import Optional
import hashlib
import json

class ResponseCache:
    """Simple␣cache␣for␣tool␣responses."""

    def __init__(self, ttl_seconds: int = 300):
        self.cache: Dict[str, Tuple[Any, datetime]] = {}
        self.ttl = timedelta(seconds=ttl_seconds)

    def _make_key(self, tool_name: str, args: dict) -> str:
        """Create␣cache␣key␣from␣tool␣and␣arguments."""
        args_str = json.dumps(args, sort_keys=True)
        return hashlib.sha256(
            f"{tool_name}:{args_str}".encode()
        ).hexdigest()

    def get(self, tool_name: str, args: dict) -> Optional[Any]:
        """Get␣cached␣response␣if␣exists␣and␣not␣expired."""
        key = self._make_key(tool_name, args)

        if key not in self.cache:
            return None

        value, timestamp = self.cache[key]

        # Check if expired
        if datetime.now() - timestamp > self.ttl:
            del self.cache[key]
            return None

        return value

    def put(self, tool_name: str, args: dict, result: Any):
        """Store␣response␣in␣cache."""
        key = self._make_key(tool_name, args)
        self.cache[key] = (result, datetime.now())

    def invalidate(self, tool_name: str = None):
        """Invalidate␣cache␣entries."""
        if tool_name is None:
            self.cache.clear()
        else:
            # Remove all entries for this tool
            keys_to_remove = [
                k for k in self.cache.keys()
                if k.startswith(tool_name)
            ]
```


© Dr. Segal Yoram - כל הזכויות שמורות

4.10.2 ביצוע מקבילי

כאשר צריך להפעיל מספר כלים ללא תלות ביניהם, ביצוע מקבילי יכול לחסוך זמן משמעותי:

ביצוע מקבילי של מספר כלים

```
async def execute_tools_parallel(
    client: MultiServerClient,
    tool_calls: List[Tuple[str, dict]]
) -> List[dict]:
    """Execute␣multiple␣tool␣calls␣in␣parallel."""
    tasks = [
        safe_tool_call(client, tool_name, args)
        for tool_name, args in tool_calls
    ]

    # Wait for all to complete
    results = await asyncio.gather(*tasks, return_exceptions=True)

    # Process results
    processed = []
    for i, result in enumerate(results):
        if isinstance(result, Exception):
            processed.append({
                "success": False,
                "error": str(result),
                "tool": tool_calls[i][0]
            })
        else:
            processed.append(result)

    return processed
```

4.11 סיכום הפרק

בפרק זה בחנו לעומק את ארכיטקטורת לקוח MCP:

- הבנו את ההקשר ההיסטורי של מודל לקוח-שרת והמשמעות שלו בעולם של סוכני LLM
- הגדרנו מודל פורמלי למצב הלקוח באמצעות מתמטיקה: $C = (S, T, M, R)$
- בחנו את רכיבי הליבה: מנהל סשנים, רישום כלים, ומטפל תעבורה
- ראינו תרשימים של הארכיטקטורה הפנימית וטופולוגיית חיבור רב-שרתית
- למדנו על זרימת הודעות ומחזור חיי בקשה-תשובה

- יישמנו לקוח מלא ב-Python עם תמיכה במספר שרתים
- הבנו את חלוקת האחריות בין לקוח לשרת
- בחנו טכניקות לטיפול בשגיאות, מטמון, וביצוע מקבילי

בפרק הבא נעבור לפרקטיקה מעמיקה יותר ונבנה לקוח MCP מלא מאפס, עם כל התכונות המתקדמות שלמדנו.

כל הזכויות שמורות - Dr. Segal Yoram ©

5 בניית לקוחות MCP — מדריך מעשי

5.1 על בונים ומשתמשים

במשך רוב ההיסטוריה האנושית, היה הבדל ברור בין יצרנים למשתמשים. הנפח היה מייצר כלי נשק שאחרים ישתמשו בהם. הבנאי היה בונה בתים שאחרים יגורו בהם. החלוקה הזו הייתה ברורה ומובנת מאליה.

אבל בעולם התוכנה, הגבולות הללו מטושטשים. כל מתכנת הוא גם בונה וגם משתמש. אנו משתמשים בספריות שאחרים כתבו, ובו זמנית בונים כלים שאחרים ישתמשו בהם. המפתח שכותב לקוח MCP היום, ייתכן שימצא את עצמו מחר כותב שרת MCP שאותו לקוח יתחבר אליו[16].

בפרק זה נעבור את המסע מרעיון ללקוח עובד. נתחיל מהבסיס המוחלט — התקנת ספריה — ונסיים עם לקוח מלא המסוגל לנהל מספר שרתים, להפעיל כלים, ולטפל בשגיאות. זהו מסע מעשי, והדרך הטובה ביותר ללמוד היא לכתוב קוד.

הפילוסופיה שלנו תהיה פשוטה: נתחיל עם הקוד הפשוט ביותר שעובד, ונוסיף שכבות של מורכבות רק כאשר הן נחוצות. לא נבנה מערכת מופשטת ומורכבת מראש. במקום זאת, נתחיל עם משהו פשוט ועובד, ונלמד מהטעויות שלנו.

5.2 הקמת סביבת הפיתוח

לפני שנתחיל לכתוב קוד, עלינו להכין את סביבת העבודה שלנו. זה דומה להכנת מטבח לפני בישול — כלים נכונים במקום הנכון הופכים את כל התהליך לקל יותר.

5.2.1 התקנת Python SDK

ה-SDK הרשמי של MCP זמין דרך pip, מנהל החבילות של Python. ההתקנה פשוטה, אך יש כמה נקודות שחשוב להבין[17]:

התקנת MCP SDK

```
# Install the core MCP SDK
pip install mcp

# For development and testing, also install:
pip install httpx       # HTTP transport support
pip install anyio       # Async utilities and abstractions
pip install pytest      # For testing our client
pip install pytest-asyncio  # Async test support
```

ה-SDK מספק את כל מה שצריך לבניית לקוח: מחלקות לניהול חיבורים, פרוטוקול JSON-RPC, ותמיכה בסוגי תעבורה שונים. הוא בנוי כולו על Python אסינכרוני (async/await), מה שמאפשר ביצועים גבוהים וניהול יעיל של מספר חיבורים במקביל.

5.2.2 מבנה הפרויקט המומלץ

ארגון נכון של הפרויקט הוא קריטי. מבנה טוב הופך את הקוד לקריא יותר, את התחזוקה לקלה יותר, ואת ההרחבות העתידיות לאפשריות:

מבנה ספריות הפרויקט

```
my_mcp_client/├──
 __init__.py├──
 client.py              # Main client implementation├──
 session_manager.py     # Session lifecycle management├──
 tool_executor.py       # Tool execution and error handling├──
 config_loader.py       # Configuration file parsing├──
 exceptions.py          # Custom exception classes├──
 main.py               # CLI entry point├──
 config/│└──
    servers.json       # Server configurations└──
 tests/├──
     __init__.py├──
     test_client.py├──
     test_tools.py└──
     fixtures/└──
         mock_server.py
```

כל קובץ אחראי על תחום ספציפי. זה עוקב אחר עיקרון האחריות היחידה (Single Responsibility Principle) — כל מודול עושה דבר אחד ועושה אותו טוב.

5.3 לקוח מינימלי — הצעד הראשון

נתחיל עם הלקוח הפשוט ביותר האפשרי. זהו לקוח שיכול להתחבר לשרת אחד, לקבל את רשימת הכלים, ולהדפיס אותה. זה הכל. אבל זה מספיק כדי להבין את העקרונות הבסיסיים:

כל הזכויות שמורות - Dr. Segal Yoram ©

לקוח MCP מינימלי

```
"""
minimal_client.py␣-␣The␣simplest␣possible␣MCP␣client
Connects␣to␣a␣server␣and␣lists␣available␣tools.
"""
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    """Minimal␣MCP␣client␣example."""
    # Step 1: Define the server we want to connect to
    server_params = StdioServerParameters(
        command="python",
        args=["example_server.py"],
        env=None  # Optional environment variables
    )

    # Step 2: Connect using stdio transport
    # The context managers ensure proper cleanup
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            # Step 3: Initialize the connection
            # This performs the handshake with the server
            await session.initialize()

            # Step 4: List available tools
            tools_response = await session.list_tools()

            print(f"Connected␣to␣server!")
            print(f"Available␣tools:␣{len(tools_response.tools)}")
            print()

            for tool in tools_response.tools:
                print(f"Tool:␣{tool.name}")
                print(f"␣␣Description:␣{tool.description}")
                if hasattr(tool, 'inputSchema'):
                    print(f"␣␣Parameters:␣{tool.inputSchema}")
                print()

if __name__ == "__main__":
    asyncio.run(main())
```

הקוד הזה מדגים את שלושת הרכיבים המרכזיים של כל לקוח MCP:

1. **הגדרת פרמטרי השרת** — מי הוא השרת ואיך להתחבר אליו

2. **יצירת החיבור** — באמצעות stdio או פרוטוקול אחר

3. **אתחול הסשן** — ביצוע לחיצת יד (handshake) עם השרת

5.4 הפעלת כלים — הליבה האמיתית

היכולת להפעיל כלים היא הליבה של כל לקוח MCP. בלי זה, הלקוח הוא רק צופה פסיבי. עם זה, הוא הופך לסוכן פעיל שיכול לשנות את העולם סביבו:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
"""
tool_caller.py␣-␣Demonstrates␣tool␣calling
"""
from mcp import ClientSession
from mcp.types import TextContent, ImageContent, EmbeddedResource

async def call_tool_example(session: ClientSession):
    """Example␣of␣calling␣an␣MCP␣tool␣with␣error␣handling."""
    try:
        # Call a tool with arguments
        result = await session.call_tool(
            name="read_file",
            arguments={
                "path": "/home/user/document.txt"
            }
        )

        # Process the result based on content type
        # MCP tools can return text, images, or resources
        for content in result.content:
            if isinstance(content, TextContent):
                print(f"File␣contents:")
                print(content.text)
            elif isinstance(content, ImageContent):
                print(f"Received␣image:␣{content.mimeType}")
                # Could save to file or display
            elif isinstance(content, EmbeddedResource):
                print(f"Received␣resource:␣{content.resource.uri}")

        # Check if the tool reported an error
        if result.isError:
            print(f"Tool␣error:␣{result.content}")
            return None

        return result

    except Exception as e:
        print(f"Error␣calling␣tool:␣{e}")
        return None

async def demonstrate_multiple_tools(session: ClientSession):
    """Show␣calling␣multiple␣tools␣in␣sequence."""
    # First tool: list files in directory
    files_result = await session.call_tool(
        name="list_directory",
        arguments={"path": "/home/user/documents"}
```



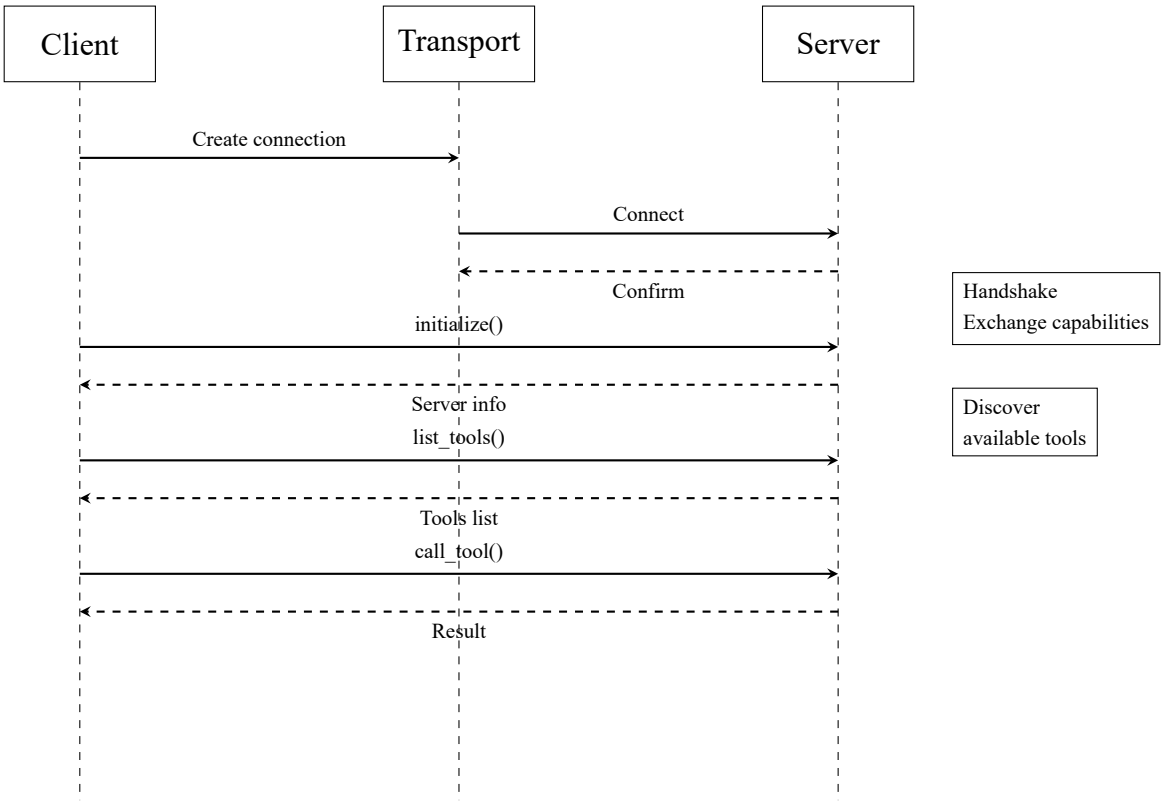
```
    # Process results and call second tool based on output
    if not files_result.isError:
```

שימו לב לשני דברים חשובים בקוד הזה:

- **טיפול בסוגי תוכן שונים** — כלי יכול להחזיר טקסט, תמונה, או משאב מוטמע
- **בדיקת שגיאות** — כל קריאה לכלי עלולה להיכשל, ועלינו לטפל בכך

5.5 תרשים רצף של אתחול לקוח

להבנה טובה יותר של תהליך האתחול, נסתכל על תרשים הרצף המלא. זה מראה את כל השלבים מרגע יצירת הלקוח ועד לקריאה הראשונה לכלי:



התרשים מראה שלושה שלבים ברורים: **התחברות**, **אתחול**, ו**שימוש**. כל שלב חייב להצליח לפני שניתן לעבור לשלב הבא.

5.6 לולאת הסוכן המלאה

כעת נחבר את כל החלקים יחד ללולאת סוכן מלאה. זה המקום שבו MCP באמת מתחיל לזרוח — שילוב של מודל שפה עם גישה לכלים:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
"""
full_agent.py␣-␣Complete␣agent␣implementation␣with␣MCP␣tools
"""
from anthropic import Anthropic
from mcp import ClientSession
from typing import List, Dict, Any
import json

class MCPAgent:
    """
␣␣␣␣Full␣MCP␣agent␣with␣tool␣calling␣capabilities.
␣␣␣␣Integrates␣Claude␣with␣MCP␣tools␣for␣autonomous␣operation.
␣␣␣␣"""

    def __init__(self, session: ClientSession, api_key: str):
        """Initialize␣agent␣with␣MCP␣session␣and␣Anthropic␣client."""
        self.session = session
        self.anthropic = Anthropic(api_key=api_key)
        self.tools = []
        self.messages = []

    async def initialize(self):
        """Load␣available␣tools␣from␣the␣MCP␣server."""
        response = await self.session.list_tools()

        # Convert MCP tool format to Anthropic tool format
        self.tools = [
            {
                "name": tool.name,
                "description": tool.description or "No␣description",
                "input_schema": tool.inputSchema or {
                    "type": "object",
                    "properties": {}
                }
            }
            for tool in response.tools
        ]

        print(f"Initialized␣with␣{len(self.tools)}␣tools")

    async def run(self, task: str, max_iterations: int = 10) -> str:
        """
␣␣␣␣␣␣␣␣Execute␣a␣task␣using␣the␣agent␣loop.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣task:␣The␣task␣description␣from␣the␣user
␣␣␣␣␣␣␣␣␣␣␣␣max_iterations:␣Maximum␣number␣of␣tool␣calls␣to␣prevent␣
loops

␣␣␣␣␣␣␣␣Returns:
```

הקוד הזה מממש לולאת סוכן מלאה עם מספר תכונות חשובות:

- **מגבלת איטרציות** — מונע לולאות אינסופיות

- **טיפול בשגיאות** — כל קריאה לכלי עטופה ב-try-catch

- **היסטוריית הודעות** — שומר את כל ההקשר לאורך הדיאלוג

- **המרת פורמטים** — מתרגם בין פורמט MCP לפורמט Anthropic

5.7 טיפול בתעבורת HTTP

עד כה התמקדנו בתעבורת stdio, שמתאימה לשרתים מקומיים. אבל מה אם נרצה להתחבר לשרת מרוחק? לשם כך נשתמש בתעבורת HTTP:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
"""
http_client.py␣-␣Connect␣to␣remote␣MCP␣servers␣via␣HTTP
"""
from mcp.client.sse import sse_client
from mcp import ClientSession
import httpx

async def connect_http_server(url: str, api_key: str = None):
    """
␣␣␣␣Connect␣to␣a␣remote␣MCP␣server␣via␣Server-Sent␣Events␣(SSE).

␣␣␣␣Args:
␣␣␣␣␣␣␣␣url:␣Server␣URL␣(e.g.,␣https://mcp.example.com/sse)
␣␣␣␣␣␣␣␣api_key:␣Optional␣API␣key␣for␣authentication
␣␣␣␣"""
    # Prepare headers with authentication if provided
    headers = {}
    if api_key:
        headers["Authorization"] = f"Bearer␣{api_key}"

    # Create HTTP client with timeout and headers
    async with httpx.AsyncClient(
        timeout=30.0,
        headers=headers
    ) as http_client:
        # Connect using SSE transport
        async with sse_client(url, http_client) as (read, write):
            async with ClientSession(read, write) as session:
                # Initialize connection
                await session.initialize()

                # List available tools
                tools = await session.list_tools()
                print(f"Connected␣to␣{url}")
                print(f"Available␣tools:␣{len(tools.tools)}")

                # Can now use the session just like stdio
                return session

async def multi_transport_example():
    """Example␣of␣connecting␣to␣multiple␣servers␣via␣different␣
transports."""
    from mcp.client.stdio import stdio_client
    from mcp import StdioServerParameters

    # Connect to local server via stdio
    local_params = StdioServerParameters(
        command="python",
        args=["local_server.py"]
    )
```

שימו לב לנקודות החשובות:

- **אימות** — מעבירים API key דרך כותרות HTTP

- **טיימאאוטים** — מגדירים זמני המתנה מקסימליים

- **ריבוי חיבורים** — אפשר להתחבר למספר שרתים במקביל

5.8 ניהול קונפיגורציה

לקוח אמיתי צריך לטעון הגדרות מקובץ, לא מקוד קשיח. זה מאפשר גמישות ושימוש חוזר:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
"""
config_loader.py␣-␣Load␣MCP␣server␣configurations
"""
import json
from pathlib import Path
from dataclasses import dataclass
from typing import Optional, Dict

@dataclass
class ServerConfig:
    """Configuration␣for␣a␣single␣MCP␣server."""
    name: str
    transport: str  # "stdio" or "sse"
    command: Optional[str] = None
    args: Optional[list[str]] = None
    env: Optional[Dict[str, str]] = None
    url: Optional[str] = None
    api_key: Optional[str] = None

class ConfigLoader:
    """Load␣and␣parse␣MCP␣server␣configurations."""

    @staticmethod
    def load_servers(config_path: str) -> list[ServerConfig]:
        """
␣␣␣␣␣␣␣␣Load␣server␣configurations␣from␣JSON␣file.

␣␣␣␣␣␣␣␣Expected␣format:
␣␣␣␣␣␣␣␣{
␣␣␣␣␣␣␣␣␣␣␣␣"mcpServers":␣{
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"filesystem":␣{
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"transport":␣"stdio",
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"command":␣"python",
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"args":␣["fs_server.py"],
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"env":␣{"DATA_DIR":␣"/data"}
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣},
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"api-server":␣{
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"transport":␣"sse",
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"url":␣"https://api.example.com/mcp",
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣"api_key":␣"secret-key"
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣}
␣␣␣␣␣␣␣␣␣␣␣␣}
␣␣␣␣␣␣␣␣}
␣␣␣␣␣␣␣␣"""
        path = Path(config_path)
        if not path.exists():
            raise FileNotFoundError(
                f"Configuration␣file␣not␣found:␣{config_path}"
            )
```

5.9 דוגמה מלאה: לקוח CLI מתוחכם

נסיים עם דוגמה מלאה ללקוח CLI שמשלב את כל מה שלמדנו:

כל הזכויות שמורות - Dr. Segal Yoram ©

לקוח CLI מלא עם ניהול שרתים

```
"""
mcp_cli.py␣-␣Complete␣CLI␣client␣for␣MCP
Usage:␣python␣mcp_cli.py␣--config␣servers.json␣"Read file.txt"
"""
import asyncio
import argparse
import sys
from contextlib import AsyncExitStack
from pathlib import Path

# Import our modules
from config_loader import ConfigLoader, ServerConfig
from full_agent import MCPAgent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp.client.sse import sse_client
import httpx

class MCPCLIClient:
    """CLI␣client␣that␣manages␣multiple␣MCP␣servers."""

    def __init__(self, config_path: str, anthropic_key: str):
        self.config_path = config_path
        self.anthropic_key = anthropic_key
        self.sessions: Dict[str, ClientSession] = {}

    async def connect_all_servers(self, stack: AsyncExitStack):
        """Connect␣to␣all␣configured␣servers."""
        configs = ConfigLoader.load_servers(self.config_path)

        for config in configs:
            if not ConfigLoader.validate_config(config):
                print(f"Invalid␣config␣for␣{config.name},␣skipping")
                continue

            try:
                session = await self._connect_server(config, stack)
                await session.initialize()
                self.sessions[config.name] = session
                print(f"Connected␣to:␣{config.name}")
            except Exception as e:
                print(f"Failed␣to␣connect␣to␣{config.name}:␣{e}")

    async def _connect_server(
        self,
        config: ServerConfig,
        stack: AsyncExitStack
    ) -> ClientSession:
        """Connect␣to␣a␣single␣server␣based␣on␣its␣configuration."""
        if config.transport == "stdio":
```

5.10 טיפול בשגיאות ונתיבי התאוששות

לקוח אמין חייב לטפל בשגיאות בצורה חכמה. הנה דוגמה למערכת טיפול בשגיאות מקיפה:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
"""
error_handling.py␣-␣Robust␣error␣handling␣for␣MCP␣clients
"""
from enum import Enum
from typing import Optional, Callable
import asyncio

class ErrorSeverity(Enum):
    """Severity␣levels␣for␣errors."""
    TRANSIENT = "transient"  # Temporary, retry possible
    PERMANENT = "permanent"  # Cannot recover
    FATAL = "fatal"          # System must shut down

class MCPError(Exception):
    """Base␣exception␣for␣MCP␣errors."""
    def __init__(self, message: str, severity: ErrorSeverity):
        super().__init__(message)
        self.severity = severity

class RetryStrategy:
    """Implements␣exponential␣backoff␣retry␣strategy."""

    def __init__(
        self,
        max_retries: int = 3,
        base_delay: float = 1.0,
        max_delay: float = 30.0
    ):
        self.max_retries = max_retries
        self.base_delay = base_delay
        self.max_delay = max_delay

    async def execute_with_retry(
        self,
        func: Callable,
        *args,
        **kwargs
    ):
        """Execute␣a␣function␣with␣exponential␣backoff␣retry."""
        for attempt in range(self.max_retries):
            try:
                return await func(*args, **kwargs)
            except Exception as e:
                if attempt == self.max_retries - 1:
                    raise

                delay = min(
                    self.base_delay * (2 ** attempt),
                    self.max_delay
                )
```

5.11 סיכום הפרק

בפרק זה עברנו מסע שלם בבניית לקוח MCP פונקציונלי. התחלנו מהתקנת הספריות הבסיסיות והגענו ללקוח מלא עם יכולות מתקדמות[16], [17].

מה למדנו:

- **התקנה והגדרה** — איך להכין סביבת פיתוח עם SDK של MCP
- **לקוח מינימלי** — היסודות של חיבור לשרת ורישום כלים
- **הפעלת כלים** — איך לקרוא לכלים ולעבד את התוצאות
- **לולאת סוכן** — שילוב מודל שפה עם כלים MCP ללולאה אוטונומית
- **תעבורות שונות** — תמיכה ב-stdio ו-SSE/HTTP
- **ניהול קונפיגורציה** — טעינת הגדרות מקבצי JSON
- **לקוח CLI מלא** — יישום מעשי שניתן להריץ משורת הפקודה
- **טיפול בשגיאות** — אסטרטגיות התאוששות ונסיונות חוזרים

הכלים שפיתחנו בפרק זה הם בסיס מוצק לכל יישום סוכנים. בפרק הבא נעמיק ביכולות מתקדמות: ניהול מספר שרתים במקביל, אופטימיזציית ביצועים, ואבטחת התקשורת.

הקוד שכתבנו אינו רק דוגמאות לימודיות — זהו קוד ייצור אמיתי שניתן לקחת ולהשתמש בו כבסיס לפרויקטים משלכם. התחילו מהקוד הפשוט, הבינו אותו היטב, ואז הרחיבו אותו בהתאם לצרכים שלכם.

זכרו: הדרך הטובה ביותר ללמוד היא לעשות. קחו את הדוגמאות מהפרק הזה, הריצו אותן, שנו אותן, שברו אותן. רק דרך ניסוי וטעייה תבינו באמת איך הכל עובד.

כל הזכויות שמורות - Dr. Segal Yoram ©

6 יכולות מתקדמות בלקוח MCP

6.1 על המורכבות והתוחכה

כאשר הומו סאפיינס החל לבנות כלים לפני כ-5.2 מיליון שנה, הכלים הראשונים היו פשוטים — אבן משובבת, מקל מחודד. אך עם הזמן, התוחכה גדלה. מחרט עור דרש הבנה של חיתוך וקצוות. צייד חניתות דרש תכנון של משקולות ואווירודינמיקה. הקשת והחץ היו מערכת מורכבת של חומרים שונים, כל אחד עם תפקיד משלו.

בדיוק כך, סוכני בינה מלאכותית מתחילים פשוט ומתפתחים להיות מתוחכמים יותר ויותר[18]. בפרקים הקודמים בנינו לקוח MCP בסיסי — האבן המשובבת של עולם הסוכנים. כעת נתקדם לכלים מתוחכמים יותר: יכולות המאפשרות לסוכנים להפוך למערכות אוטונומיות אמיתיות.

המעבר מלקוח פשוט ללקוח מתקדם אינו רק שאלה של הוספת תכונות. זהו שינוי פרדיגמה. לקוח פשוט הוא סביל — הוא שולח בקשות ומקבל תשובות. לקוח מתקדם הוא אקטיבי — הוא מנהל מספר חיבורים במקביל, מאזין לשינויים בזמן אמת, מאפשר לשרתים לבקש שירותים ממנו, ומטפל בכשלים בחן. זוהי הרמה הבאה של אוטונומיה.

6.2 דגימה — כאשר השרת מבקש ממך לחשוב

דגימה (Sampling) היא אחת היכולות המרתקות ביותר בפרוטוקול MCP. כדי להבין את המהפכה שלה, נחשוב על ארכיטקטורת לקוח-שרת מסורתית. בדרך כלל, הלקוח מתקשר עם השרת בכיוון אחד: הלקוח שואל, השרת עונה. השרת אינו יכול לבקש מהלקוח להפעיל את המודל שלו — הוא פשוט מחזיר מידע.

דגימה מהפכת את המודל הזה[2]. עם דגימה, השרת יכול לבקש מהלקוח להפעיל את מודל השפה שלו ולחזור עם תוצאה. זה כמו שהכלים שלך פתאום מבקשים ממך לחשוב בשבילם — יצירת זרימה דו-כיוונית של אינטליגנציה.

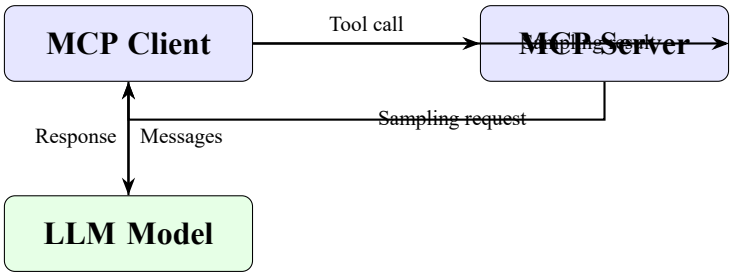
6.2.1 מתי משתמשים בדגימה?

שקלו תרחישים אלה שבהם דגימה מועילה:

- **החלטות מורכבות בשרת** — השרת צריך לבחור בין מספר אפשרויות על סמך הבנת הקשר טבעי
- **יצירת תוכן מותאם אישית** — השרת רוצה להחזיר תשובה שמותאמת לסגנון המשתמש
- **תרגום והסקה** — השרת מעבד נתונים ודורש יכולת הבנת שפה טבעית
- **סינון וסיכום** — השרת מחזיר נתונים גולמיים ומבקש מהמודל לסכם אותם

6.2.2 הזרימה הדו-כיוונית

הדיאגרמה הבאה ממחישה את הזרימה המורכבת של דגימה:



© Dr. Segal Yoram - כל הזכויות שמורות

6.2.3 מימוש דגימה בלקוח

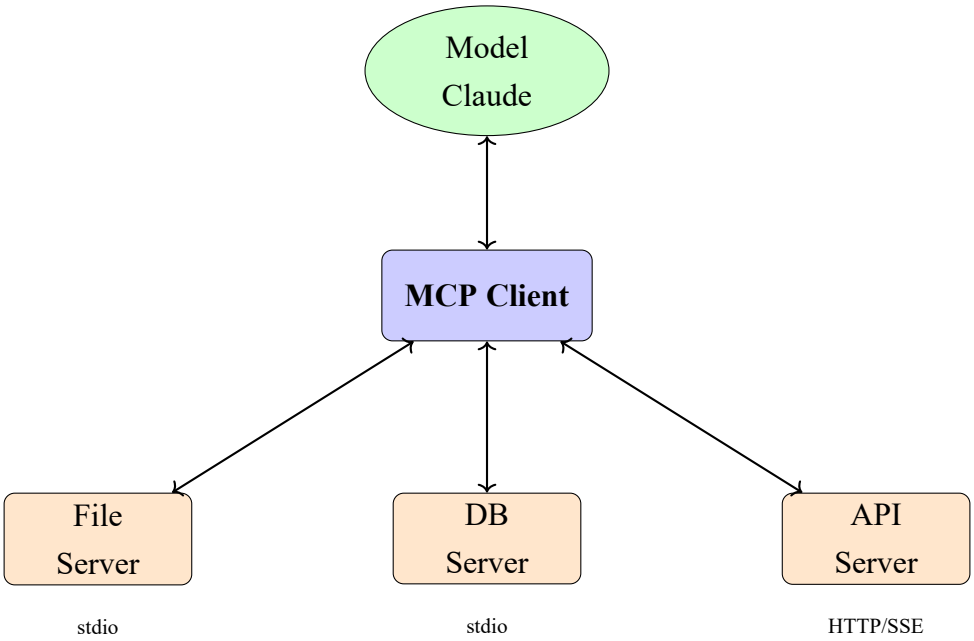
מימוש יכולת דגימה

```
from mcp.types import SamplingMessage, CreateMessageResult
from anthropic import Anthropic
from typing import Optional

class SamplingCapableClient:
    """MCP␣client␣with␣sampling␣capability."""

    def __init__(self, session, api_key: str):
        self.session = session
        self.anthropic = Anthropic(api_key=api_key)
        self.sampling_enabled = True

    async def handle_sampling_request(
        self,
        messages: list[SamplingMessage],
        model_preferences: Optional[dict] = None,
        max_tokens: int = 1000
    ) -> CreateMessageResult:
        """Handle␣sampling␣request␣from␣MCP␣server."""
        # Convert MCP messages to Anthropic format
        anthropic_messages = []
        system_prompt = None

        for msg in messages:
            if msg.role == "system":
                system_prompt = msg.content.text
            else:
                anthropic_messages.append({
                    "role": msg.role,
                    "content": msg.content.text
                })

        # Select appropriate model
        model = self._select_model(model_preferences)

        # Call the LLM
        response = self.anthropic.messages.create(
            model=model,
            max_tokens=max_tokens,
            system=system_prompt,
            messages=anthropic_messages
        )

        # Return result in MCP format
        return CreateMessageResult(
            role="assistant",
            content={
                "type": "text",
```

6.3 ניהול מרובה שרתים

בעולם המודרני, סוכן אמיתי אינו מתחבר לשרת אחד בלבד. הוא צריך לגשת למספר מקורות נתונים ויכולות במקביל — שרת אחד לניהול קבצים, אחר למסדי נתונים, שלישי ל-API חיצוניים. ניהול מספר שרתים הוא אתגר ארכיטקטוני מרכזי.

6.3.1 טופולוגיית רב-שרתים



כל הזכויות שמורות - Dr. Segal Yoram ©

6.3.2 מחלקת ניהול חיבורים

מנהל חיבורי שרתים

```
from dataclasses import dataclass
from typing import Optional, Dict, List
import asyncio

@dataclass
class ServerConnection:
    """Represents␣connection␣to␣an␣MCP␣server."""
    name: str
    session: ClientSession
    tools: List[dict]
    resources: List[dict]
    prompts: List[dict]
    status: str = "connected"
    latency_ms: float = 0.0
    error_count: int = 0

class MultiServerManager:
    """Manage␣multiple␣MCP␣server␣connections."""

    def __init__(self):
        self.connections: Dict[str, ServerConnection] = {}
        self.tool_registry: Dict[str, str] = {}
        self.resource_registry: Dict[str, str] = {}
        self.lock = asyncio.Lock()

    async def connect_server(
        self,
        name: str,
        params
    ) -> ServerConnection:
        """Connect␣to␣a␣new␣MCP␣server."""
        async with self.lock:
            if name in self.connections:
                raise ValueError(f"Server␣{name}␣already␣connected")

        # Establish connection
        async with stdio_client(params) as (read, write):
            session = ClientSession(read, write)
            await session.initialize()

            # Fetch capabilities
            tools_resp = await session.list_tools()
            resources_resp = await session.list_resources()
            prompts_resp = await session.list_prompts()
```



```
            # Register tools and resources
            for tool in tools_resp.tools:
                self.tool_registry[tool.name] = name
```

6.4 מנויים למשאבים

מנויים למשאבים (Resource Subscriptions) מאפשרים ללקוח לקבל עדכונים אוטומטיים כאשר משאב משתנה. במקום לשאול שוב ושוב "האם משהו השתנה?", הלקוח רושם עניין במשאב והשרת מודיע לו על כל שינוי.

זוהי תכונה קריטית למערכות זמן אמת. דמיינו סוכן המעקב אחר שינויים בתיקיית קבצים, או סוכן המקבל עדכונים על שורות חדשות במסד נתונים. מבלי מנויים, הסוכן היה צריך לבדוק כל הזמן — בזבוז משאבים. עם מנויים, השרת דואג להודיע רק כאשר באמת יש שינוי.

כל הזכויות שמורות - Dr. Segal Yoram ©

6.4.1 מימוש מערכת מנויים

מנהל מנויים למשאבים

```
from typing import Callable, Awaitable
import asyncio

class ResourceSubscriptionManager:
    """Manage␣resource␣subscriptions␣and␣notifications."""

    def __init__(self, manager: MultiServerManager):
        self.manager = manager
        self.subscriptions: Dict[str, List[Callable]] = {}
        self.notification_queue = asyncio.Queue()

    async def subscribe(
        self,
        resource_uri: str,
        callback: Callable[[str, dict], Awaitable[None]]
    ):
        """Subscribe␣to␣resource␣updates."""
        # Find which server owns this resource
        server_name = self.manager.resource_registry.get(
            resource_uri
        )
        if not server_name:
            raise ValueError(
                f"Resource␣{resource_uri}␣not␣found"
            )

        # Add callback to subscriptions
        if resource_uri not in self.subscriptions:
            self.subscriptions[resource_uri] = []
        self.subscriptions[resource_uri].append(callback)

        # Tell server we want to subscribe
        conn = self.manager.connections[server_name]
        await conn.session.subscribe_resource(resource_uri)

    async def unsubscribe(
        self,
        resource_uri: str,
        callback: Optional[Callable] = None
    ):
        """Unsubscribe␣from␣resource␣updates."""
        if resource_uri not in self.subscriptions:
            return

        if callback:
            # Remove specific callback
            self.subscriptions[resource_uri].remove(callback)
        else:
```

6.5 התראות התקדמות

התראות התקדמות (Progress Notifications) מאפשרות לשרת לדווח על התקדמות פעולות ארוכות. זה חיוני לחוויית משתמש טובה — במקום שהמשתמש יתהה האם המערכת קפאה, הוא רואה בדיוק מה קורה.

כל הזכויות שמורות - Dr. Segal Yoram ©

```
from dataclasses import dataclass
from enum import Enum

class ProgressState(Enum):
    """Progress␣notification␣states."""
    STARTED = "started"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"

@dataclass
class ProgressNotification:
    """Progress␣update␣notification."""
    operation_id: str
    state: ProgressState
    progress: float  # 0.0 to 1.0
    message: str
    details: Optional[dict] = None

class ProgressTracker:
    """Track␣progress␣of␣long-running␣operations."""

    def __init__(self):
        self.operations: Dict[str, ProgressNotification] = {}
        self.callbacks: Dict[str, List[Callable]] = {}

    def start_operation(
        self,
        operation_id: str,
        message: str
    ):
        """Start␣tracking␣a␣new␣operation."""
        notification = ProgressNotification(
            operation_id=operation_id,
            state=ProgressState.STARTED,
            progress=0.0,
            message=message
        )
        self.operations[operation_id] = notification
        self._notify(operation_id, notification)

    def update_progress(
        self,
        operation_id: str,
        progress: float,
        message: str,
        details: Optional[dict] = None
    ):
        """Update␣operation␣progress."""
        if operation_id not in self.operations:
```

6.6 אסטרטגיות התאוששות משגיאות

במערכות מבוזרות עם חיבורים רבים, **כשלים הם בלתי נמנעים**. שרת יכול ליפול, חיבור רשת יכול להתנתק, פעולה יכולה להיכשל. לקוח מתקדם חייב להיות עמיד בפני כשלים ולדעת להתאושש.

6.6.1 טקסונומיה של שגיאות

ישנם מספר סוגי שגיאות שלקוח MCP צריך לטפל בהם:

- **שגיאות חיבור** — השרת לא זמין או החיבור נכשל
- **שגיאות זמן קצוב** — הפעולה לקחה יותר מדי זמן
- **שגיאות פרוטוקול** — הודעה לא תקינה או גרסה לא תואמת
- **שגיאות ביצוע** — הכלי או המשאב נכשל בצד השרת
- **שגיאות זמניות** — עומס או בעיה ארעית

6.6.2 מודל מתמטי לניסיונות חוזרים

נשתמש ב**נסיגה אקספוננציאלית** (Exponential Backoff) עם רעש למניעת "אפקט הרעם" (Thundering Herd):

$$T_n = \min(T_{\max}, T_0 \cdot 2^n) + U(0, J)$$

כאשר:

- T_n — זמן ההמתנה בניסיון ה-n
- T_0 — זמן המתנה בסיסי (למשל 1 שנייה)
- $T_{\max}$ — זמן המתנה מקסימלי (למשל 06 שניות)
- $U(0, J)$ — רעש אחיד בטווח $[0, J]$ (למשל $J = 1$ שנייה)
- n — מספר הניסיון (מתחיל מ-0)

כל הזכויות שמורות - Dr. Segal Yoram ©

```python
import asyncio
import random
from typing import TypeVar, Callable, Awaitable

T = TypeVar('T')

class RetryStrategy:
    """Implements␣exponential␣backoff␣with␣jitter."""

    def __init__(
        self,
        max_retries: int = 5,
        base_delay: float = 1.0,
        max_delay: float = 60.0,
        jitter: float = 1.0
    ):
        self.max_retries = max_retries
        self.base_delay = base_delay
        self.max_delay = max_delay
        self.jitter = jitter

    def calculate_delay(self, attempt: int) -> float:
        """Calculate␣delay␣for␣given␣attempt␣number."""
        # Exponential backoff
        delay = min(
            self.max_delay,
            self.base_delay * (2 ** attempt)
        )

        # Add jitter
        delay += random.uniform(0, self.jitter)

        return delay

    async def execute_with_retry(
        self,
        func: Callable[..., Awaitable[T]],
        *args,
        **kwargs
    ) -> T:
        """Execute␣function␣with␣retry␣logic."""
        last_exception = None

        for attempt in range(self.max_retries):
            try:
                return await func(*args, **kwargs)
            except Exception as e:
                last_exception = e

                # Check if error is retryable
```

6.7 איזון עומסים בין שרתים

כאשר יש לנו מספר שרתים המספקים יכולות דומות, נוכל לחלק את העומס ביניהם. זה משפר ביצועים ועמידות — אם שרת אחד עמוס או נכשל, אפשר לנתב בקשות לשרת אחר.

6.7.1 מודל מתמטי לאיזון עומסים

נשתמש באלגוריתם **בחירה משוקללת** על סמך זמן תגובה וזמינות:

$$w_i = \frac{1}{L_i \cdot (1 + E_i)}$$

$$P(i) = \frac{w_i}{\sum_{j=1}^{n} w_j}$$

כאשר:

- w_i — משקל השרת i
- L_i — השהיה ממוצעת של שרת i (במילישניות)
- E_i — שיעור שגיאות של שרת i (בין 0 ל-1)
- $P(i)$ — הסתברות לבחירת שרת i

כל הזכויות שמורות - Dr. Segal Yoram ©

```
import time
from typing import List, Optional
import random

class LoadBalancer:
    """Intelligent␣load␣balancer␣for␣MCP␣servers."""

    def __init__(self):
        self.servers: Dict[str, ServerStats] = {}

    def add_server(self, name: str):
        """Register␣a␣server␣for␣load␣balancing."""
        self.servers[name] = ServerStats(name)

    def select_server(
        self,
        available_servers: List[str]
    ) -> str:
        """Select␣best␣server␣using␣weighted␣selection."""
        if not available_servers:
            raise ValueError("No␣servers␣available")

        # Calculate weights for each server
        weights = []
        for server_name in available_servers:
            stats = self.servers.get(server_name)
            if not stats:
                weights.append(1.0)  # Unknown server
            else:
                weights.append(stats.calculate_weight())

        # Normalize weights to probabilities
        total = sum(weights)
        probabilities = [w / total for w in weights]

        # Weighted random selection
        return random.choices(
            available_servers,
            weights=probabilities
        )[0]

    def record_request(
        self,
        server_name: str,
        latency_ms: float,
        success: bool
```



```
        """Record␣request␣result␣for␣statistics."""
        if server_name in self.servers:
            self.servers[server_name].record_request(
```

6.8 דוגמה מלאה: לקוח מתקדם

נסיים בדוגמה מלאה המשלבת את כל היכולות המתקדמות:

כל הזכויות שמורות - Dr. Segal Yoram ©

```
class AdvancedMCPClient:
    """Full-featured␣advanced␣MCP␣client."""

    def __init__(self, anthropic_api_key: str):
        self.server_manager = MultiServerManager()
        self.subscription_manager = ResourceSubscriptionManager(
            self.server_manager
        )
        self.progress_tracker = ProgressTracker()
        self.load_balancer = LoadBalancer()
        self.retry_strategy = RetryStrategy()
        self.sampling_client = SamplingCapableClient(
            None,
            anthropic_api_key
        )

    async def initialize(self, server_configs: List[dict]):
        """Initialize␣all␣server␣connections."""
        for config in server_configs:
            params = StdioServerParameters(
                command=config["command"],
                args=config.get("args", []),
                env=config.get("env")
            )

            conn = ResilientServerConnection(
                name=config["name"],
                params=params,
                retry_strategy=self.retry_strategy
            )

            await conn.ensure_connected()
            await self.server_manager.connect_server(
                config["name"],
                params
            )
            self.load_balancer.add_server(config["name"])

    async def execute_task(self, task: str) -> str:
        """Execute␣task␣with␣all␣advanced␣features."""
        operation_id = f"task_{int(time.time())}"
        self.progress_tracker.start_operation(
            operation_id,
            f"Starting:␣{task}"
        )

        try:
            # Get all available tools
            tools = self.server_manager.get_unified_tools_list()
```

6.9 סיכום הפרק

בפרק זה חקרנו את היכולות המתקדמות של לקוחות MCP והפכנו לקוח פשוט למערכת אגנטית מתוחכמת. למדנו על:

- **דגימה** — מנגנון דו-כיווני המאפשר לשרתים לבקש שירותי LLM מהלקוח
- **ניהול רב-שרתים** — ארכיטקטורה לחיבור ותיאום מספר שרתי MCP במקביל
- **מנויים למשאבים** — מנגנון התראות לעדכונים בזמן אמת על שינויים במשאבים
- **התראות התקדמות** — מערכת מעקב ודיווח על פעולות ארוכות
- **התאוששות משגיאות** — אסטרטגיות לטיפול בכשלים וניסיונות חוזרים עם נסיגה אקספוננציאלית
- **איזון עומסים** — חלוקה חכמה של בקשות בין שרתים על סמך ביצועים

היכולות הללו הופכות את הלקוח שלנו למערכת אמיתית ועמידה לסביבות ייצור. בפרק הבא נעבור לצד השני של המשוואה — בניית שרתי MCP עם יכולות מתקדמות משלהם.

כל הזכויות שמורות - Dr. Segal Yoram ©